

Accelerated CFD Simulations

From Early Algorithms to Modern Acceleration Techniques

Msc Thesis: Literature review
Matthias Claeys

Contents

1	Introduction	1
1.1	Background and Motivation	1
2	Literature Review	3
2.1	Scope and Goal for literature review	3
2.2	Turbulent flows: simulation and modelling	3
2.3	Performance enhancing in CFD	6
2.4	Machine/Deep learning Enhanced CFD	13
3	Methodology	24
3.1	Overall Pipeline	25
3.2	CFD Simulation Setup	25
3.3	Data Preprocessing	25
3.4	Autoencoder	25
3.5	Neural ODE	25
3.6	Out-of-Distribution Detection	25
3.7	Integrated Acceleration Workflow	25
3.8	Evaluation Design	25
3.9	Implementation Details	25
	References	26

1

Introduction

1.1. Background and Motivation

Turbulent flows are seen in all sorts of natural and engineering systems, from large-scale atmospheric flows and ocean currents to industrial processes and aerodynamic applications. Capturing these turbulent multi-scale flow dynamics has proven to be a formidable challenge, both analytically, numerically, and experimentally. To ensure qualitative research and engineering solutions, an accurate representation of these turbulent dynamics is crucial, yet achieving this with a manageable computational cost has proven to be a challenging problem.

Over the past decades, Computational Fluid Dynamics (CFD) has proven itself as an effective and powerful way to simulate fluid flow, both laminar and turbulent. It has proven its use across various industries, where it has played a central role in the analysis, design, and certification of aerospace vehicles, environmental systems, industrial products, and even healthcare developments. Advances in numerical methods and high-performance computing (HPC) have ensured steady advances in reliability and accuracy of CFD, making it a critical element in modern engineering workflows [98].

1.1.1. Need for high-fidelity yet efficient simulation

Even with the technological advancements of the last decades, the computational cost of accurate CFD simulations remains a bottleneck. This is especially the case if one aims to resolve unsteady, turbulent flows in or around complex geometries. These types of simulations require a mesh of such high quality that even the smallest and most complex flow phenomena can be captured accurately [97]. This presents two problems, each of which requires a substantial amount of time to evaluate: one is fabricating the mesh through which we want to apply our CFD algorithms, and the other is considering the CFD algorithms and subsequent turbulence models. In practice, companies navigate a spectrum of methods that trade computational efficiency for accuracy, ranging from Reynolds Averaged Navier–Stokes (RANS) to Large-Eddy Simulation (LES), hybrid RANS–LES approaches, and ultimately Direct Numerical Simulation (DNS) to gain a competitive edge in the highly competitive aeronautical markets [97]. This indicates the need for frameworks that can decrease the computational costs of high-fidelity

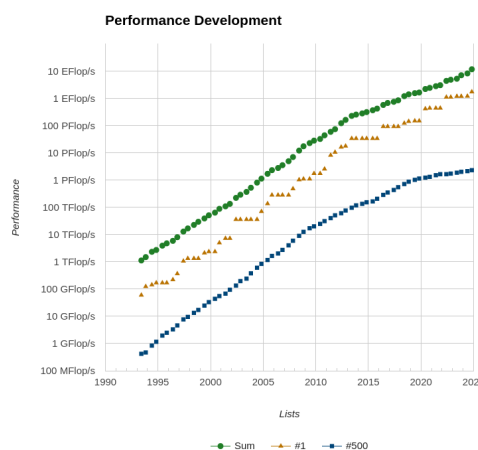


Figure 1.1: Performance trends of the TOP500 supercomputers (1993–2025), showing total performance (Sum), top system (#1), and 500th system (#500). Performance is in FLOP/s on a logarithmic scale. Adapted from TOP500 Project [104]

CFD simulations.

Compounding this is the slowing of hardware-driven performance gains. Ever since 1965, following Gordon Moore's stipulation [68], the number of transistors in a single integrated circuit has doubled approximately every two years, while maintaining a fixed cost, power, and area. However, as transistors reach the atomic scale and fabrication costs continue to rise, the classical technological driver underpinning Moore's Law for 50 years is failing and is anticipated to have flattened by 2025 [91], as shown in Figure 1.1. This 'death march' of Moore's Law [64] opens the door for new hard- and software architectures, as well as creative CFD workflows.

Besides the recent flattening of Moore's Law, researchers have been passionately innovating for several decades and finding novel ways to reduce the computational cost of CFD simulations. One way accelerated CFD simulations can be achieved is to employ advances in computer multicore architecture that have been brought to us over the last few decades [42]. These novel methods enable the use of multicore and manycore CPU (Central Processing Unit) architectures [38], as well as the use of GPU (Graphical Processing Units) to accelerate CFD solvers [113]. To achieve desirable acceleration results, these hardware methods have been paired with advanced numerical schemes which employ mesh, mesh-free, or hybrid methods to attain desirable simulation speed-ups [42].

Together with advances in computing architecture, the rapid progress of Machine Learning (ML) and Artificial Intelligence (AI) offer interesting new strategies for accelerating CFD simulations. Fluid mechanics has always been a field where an enormous amount of data needs to be dealt with. This can be derived from either experimental results or large-scale numerical simulations [12]. Together with these vast amounts of data and an increase in the ability to handle this data, ML offers a promising outlook in aiding the acceleration of modern turbulent CFD simulations.

It is in this landscape that the subject of this thesis finds itself. Bounded by the computational bottleneck created by accurate CFD simulations, and driven by technological advancements in AI technology. This thesis aims to deliver a new way of lowering the computational costs of turbulent flow simulations by employing modern AI frameworks. More specifically, this work will focus on using Neural ordinary differential equations to accelerate the time integration step. Since this is, more often than not, the most costly part in mesh based solving algorithms. In light of this context, this thesis aims to answer the following central research question and subquestions:

Research Question

How effectively can neural ordinary differential equations accelerate the time integration step in CFD solvers for turbulent flow simulations without compromising accuracy?

1. What are the key bottlenecks in conventional time integration schemes used in turbulent CFD simulations?
2. To what extent can case-specific neural ODEs maintain physical accuracy across different turbulent flow regimes, Reynolds numbers, boundary conditions, and geometric configurations (e.g., symmetric, asymmetric, irregular shapes)?
3. How sensitive is the solution accuracy of neural ODEs to network architecture and training data quality?
4. How can residuals or loss functions be used to constrain or monitor neural ODE integration quality?

2

Literature Review

2.1. Scope and Goal for literature review

This chapter provides an overview of the current knowledge and available literature on accelerated CFD simulations. As a start, [section 2.2](#) explains the most used ways in which turbulent simulations can be resolved. Next [section 2.3](#) and [section 2.4](#) will explain the various ways CFD simulations have been enhanced. Where [section 2.3](#) will introduce conventional ways of enhancing simulations, and [section 2.4](#) will introduce how current advances in machine learning and artificial intelligence are being incorporated in the current CFD landscape.

2.2. Turbulent flows: simulation and modelling

Fluid dynamics is governed by a set of coupled partial differential equations (PDE's) called the Navier-Stokes Equations (NSE). These equations are based on the assumptions of conservation of mass and momentum. These equations are shown in [2.1](#) in *Einstein notation* [\[72\]](#). Note that the equations seen in [2.1](#) display the governing equations of a fluid of which we have assumed a constant density, which means that density ρ is a material property and not something that needs to be solved for.

$$\frac{\partial u_i}{\partial x_i} = 0 \quad (2.1a)$$

$$\underbrace{\frac{\partial u_i}{\partial t}}_{\text{Local acceleration}} + \underbrace{u_j \frac{\partial u_i}{\partial x_j}}_{\text{Convective term}} = \underbrace{-\frac{1}{\rho} \frac{\partial p}{\partial x_i}}_{\text{Pressure gradient}} + \underbrace{\nu \frac{\partial^2 u_i}{\partial x_j \partial x_j}}_{\text{Viscous diffusion}} + \underbrace{g_i}_{\text{Body forces}} \quad (2.1b)$$

Where u is the 3-dimensional velocity vector, x is the position vector, p is the pressure in the fluid, ν is the kinematic viscosity of the fluid, g is the gravitational constant, t is time and finally the subscripts i and j indicate spatial directions.

These equations have proven to be a tough fluid dynamics challenge and have yet to be solved analytically. Due to their non-linearity and strong coupling, they display complex behaviour. This behaviour mainly arises from the non-linear convective term in the momentum equation. This fact is particularly pronounced for high Reynolds number flows, where small-scale turbulent structures dominate [\[72\]](#). In the current field of mathematics, solutions to the most fundamental questions regarding the NSE remain unanswered. Questions such as: do solutions exist, and are they unique [\[20\]](#)? It is due to this complexity and lack of understanding that the Clay Mathematics Institute has listed the NSE as one of the 7 Millennium Problems. Because of the absence of an analytical solution, engineers and researchers must discretise, model, and make assumptions about various parts of the NSE to find a numerical solution to these equations.

2.2.1. DNS

One way to evaluate the NSE is through a Direct Numerical Simulation (DNS). A DNS aims to capture all length and timescales of a turbulent flow directly, without modelling and without further assumptions. Furthermore, DNS must accurately resolve the temporal resolution of the turbulent flow it aims to simulate. Following Kolmogorov's microscale hypothesis [49] and the two-point correlation function [79], the smallest and largest turbulent length scales can be obtained, respectively. Using these length scales, the mesh spacing and mesh dimensions can be determined; this way, all relevant length scales can be represented in the turbulent flow field without resorting to modelling. These length scales are strongly correlated to the Reynolds numbers, which is the ratio of inertial forces and viscous forces.

$$Re = \frac{\text{Inertial forces}}{\text{Viscous forces}} = \frac{ul}{\nu} \quad (2.2)$$

Where u is the flow velocity and l is the characteristic length. At large Reynolds numbers, turbulent flows exhibit a wide range of scales, where the smallest turbulent length scales become progressively smaller as Re increases. This means that for a high Re DNS, a very fine mesh would need to be generated.

In fact, when following Kolmogorov in assuming the smallest scales of turbulence are universal and isotropic, the total number of grid points (degrees of freedom) required for a DNS scales with $Re^{9/4}$ [21]. Factoring in the necessary change in timestep to maintain a constant CFL number, the total computational effort for DNS would be equal to $\mathcal{O}(Re^3)$ [21]. This extensive computational requirement makes DNS highly accurate but prohibitively expensive for general practical use. However, it remains indispensable for fundamental turbulence research due to its ability to provide complete knowledge of the flow without empirical approximations.

2.2.2. RANS

One way to reduce the computational cost of numerical simulations of the NSE was stipulated by Reynolds [82] in 1895. In this work, he proposed decomposing the instantaneous velocity into two distinct parts: the time-averaged part and the fluctuating part.

$$u_i = \langle u_i \rangle + u'_i \quad (2.3)$$

In Equation 2.3, the instantaneous velocity u_i is decomposed into the time-averaged part $\langle u_i \rangle$ and the fluctuating component u'_i . This decomposing into time-averaged and fluctuating parts is called 'Reynolds decomposition'. Substituting Equation 2.3 into Equation 2.1 results in the incompressible Reynolds Averaged Navier Stokes (RANS) equations [69]. Because the time averaging in the Reynolds decomposition is much larger than the largest timescale of the turbulent fluctuations, the evolution of the mean flow component can be determined [83]. However, this also means that none of the turbulent fluctuations are resolved; instead, turbulent scales are modelled through a closure model, which must provide a suitable representation of the effects of these fluctuations on the mean flow.

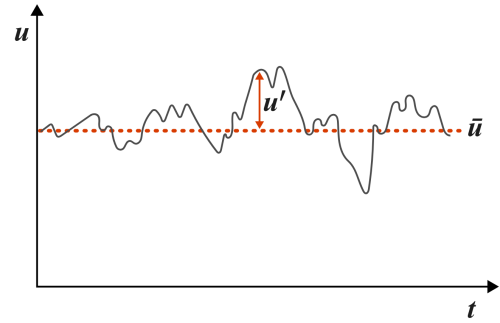


Figure 2.1: Schematic of the Reynolds decomposition for incompressible flow: the instantaneous velocity $u(t)$ (solid line) is split into its time-averaged mean $\bar{u}$ (dashed red line) and the fluctuation $u'(t) = u(t) - \bar{u}$. Adapted from [4]

After applying the substitution, Equation 2.4 can then be solved to obtain the time-averaged solution of the NSE seen in 2.4. These incompressible RANS equations are similar to the conventional NSE seen in Equation 2.1, the only difference being the addition of the divergence of the averaged products of the Reynolds stress tensor: $\frac{\partial \langle u'_i u'_j \rangle}{\partial x_j}$. To ensure accurate results, the effect of the Reynolds Stress Tensor (RST) needs to be modelled by turbulence

models. Because the time-dependent fluctuations are not being resolved, RANS simulations can be executed on grids with a larger mesh spacing, which inherently reduces the amount of required operations and thus the computational cost.

$$\frac{\partial \langle u_i \rangle}{\partial x_i} = 0 \quad (2.4a)$$

$$\frac{\partial \langle u_i \rangle}{\partial t} + \langle u_j \rangle \frac{\partial \langle u_i \rangle}{\partial x_j} = -\frac{1}{\rho} \frac{\partial \langle p \rangle}{\partial x_i} + \nu \frac{\partial^2 \langle u_i \rangle}{\partial x_j \partial x_j} + g_i - \frac{\partial \langle u'_i u'_j \rangle}{\partial x_j} \quad (2.4b)$$

To achieve accurate simulations, the effect of the RST needs to be modelled. The most used models are those of Launder and Spalding [53], Wilcox [112], Spalart and Allmaras [100], Launder, Reece, and Rodi [54] and Speziale, Sarkar, and Gatski [101].

2.2.3. LES

Another widely implemented turbulent flow framework was initially proposed by Smagorinski [95] to stabilise under-resolved weather simulations. This proposition was later applied to engineering-related flows by [23] and subsequently by Schumann [89]. In this work, the proposition was made only to resolve the large-scale turbulent structures of the flow, while modelling the effect of the small-scale structures. Because only the large-scale eddies are being resolved in this simulation, this type of simulation is called a Large Eddy Simulation (LES). The governing equations of LES are determined by applying a low-pass filter operation to Equation 2.1, which results in the governing equations seen in Equation 2.5. After the filtering operation has been applied, a sort of 'smooth' version of the velocity fluctuations is achieved, which is displayed in figure Figure 2.2.

$$\frac{\partial \bar{u}_i}{\partial x_i} = 0 \quad (2.5a)$$

$$\frac{\partial \bar{u}_i}{\partial t} + \bar{u}_j \frac{\partial \bar{u}_i}{\partial x_j} = -\frac{1}{\rho} \frac{\partial \bar{p}}{\partial x_i} + \nu \frac{\partial^2 \bar{u}_i}{\partial x_j \partial x_j} + g_i - \frac{\partial \tau_{ij}}{\partial x_j} \quad (2.5b)$$

Where $\bar{u}_i$ is the filtered velocity. The filter operation is applied through a filter kernel G , where the filter cut-off width is determined to be proportional to the grid size. As seen in Equation 2.5b, a new term arises $\frac{\partial \tau_{ij}}{\partial x_j}$, which contains the effects of the turbulent scales that the filter operation has cut off. Because most energy is dissipated from a flow in the most minor scales [72], this 'Sub Grid Scale' (SGS) tensor needs to be modelled in such a way that it artificially drains enough energy from resolved scales. Many different kinds of SGS models have been proposed, most of which utilise an 'Eddy viscosity' to equate the amount of energy drained to the strain the fluid experiences. The most used eddy viscosity models are ones like the Smagorinsky model (Smagorinski [95]), the Scale Similarity Model (Bardina, Ferziger, and Reynolds [7]), the Dynamic Smagorinsky Model (Germano et al. [30], Lilly [57]), the Wall-Adapting Local Eddy-viscosity model (Nicoud and Ducros [71]) and the Vreman model (Vreman [108]).

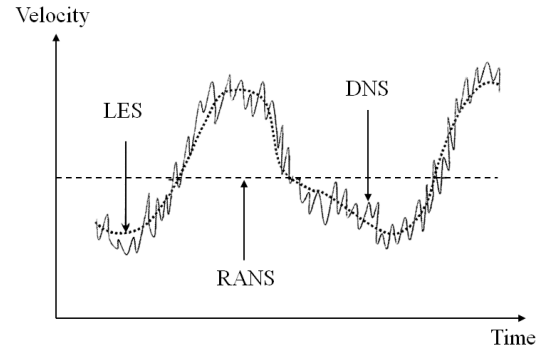


Figure 2.2: Comparison of turbulence modelling approaches based on their resolution of velocity fluctuations over time. DNS captures all spatial and temporal fluctuations, while LES resolves only the large-scale fluctuations. Adapted from [29]

Because the large-scale structures are directly resolved, LES is more accurate than the RANS approach when applied to the same flow case, as these large structures contain most of the turbulent kinetic energy and are responsible for the majority of momentum transfer and mixing. Because the accuracy of LES is determined by the filter width, which is in turn related to the grid spacing, grid refinement will lead to more accurate LES results. At the same time, refinement will also significantly

increase the computational cost of the simulation. That is why for high-accuracy flow cases, acceleration frameworks are of high necessity to keep computation cost within usable limits.

Since the concept of LES was proposed in 1963, it has undergone significant evolution and gained wider acceptance, particularly in academic and research settings. Despite the increase in computational power over the past few decades [91], the cost of performing LES remains higher than that of comparable simulations using the RANS framework. This higher cost is due to the need for LES to explicitly resolve the temporal evolution of larger turbulent eddies, which requires finer grid resolutions and more time steps, in contrast to RANS, where the equations are time-averaged and all turbulent fluctuations are modelled.

However, recent academic advancements in LES frameworks, such as those highlighted by Weymouth and Font [111], suggest that LES can be conducted at a relatively low computational cost, even with limited knowledge of the methodology. This perspective aligns with the assertion made by Piomelli [76], who stated that “LES will be used in more complex configurations and will yield reliable results for engineering analysis.”

Despite the high computational cost of LES, advancements in modelling and algorithm design will continue to reduce the cost of high-fidelity simulations. That is why, contrary to Zhiyin [115], we expect that LES will be more widely adopted in general engineering instances. Even more so with the acceleration frameworks discussed in further section, 2.3 and 2.4.

2.3. Performance enhancing in CFD

Even with the significant computational cost improvements that RANS offers and the advances in LES frameworks, there remains a need for further computational enhancements in high-fidelity applications. While RANS models reduce computational costs by utilising turbulence models, they often compromise accuracy. LES significantly improves accuracy, but remains expensive for high Reynolds number flows. This is especially the case for wall-bounded flow cases, where the computational time scales using $Re^{2.4}$ for wall-resolved meshes. In an attempt to bridge the trade-off between speed and accuracy, modern CFD advancements increasingly incorporate numerical and algorithmic techniques, such as multigrid solvers, alternative time-stepping methods, and mesh and model adaptation strategies. These methods all share the same goal of reducing computational cost while maintaining desirable computational efficiency. This section outlines key methodologies and novel frameworks designed to enhance CFD performance and maintain physical reliability. This section will build on the statements made by Hosain and Fdhila [42] and Afzal et al. [1], highlighting the current advances in the CFD acceleration landscape since 2015.

2.3.1. Numerical and Algorithmic Techniques

Numerical techniques form the foundations of modern efforts to accelerate CFD simulations while maintaining desirable accuracy. These numerical methods directly target the solver efficiency, doing so mostly by improving solver convergence rates or optimising time steps. This subsection reviews key ideas in these areas, applying a focus on their role in reducing computational cost while maintaining solution accuracy.

Multigrid

In incompressible flow simulations, the pressure field does not have its own transport equation; instead, it arises when enforcing the incompressibility condition. When the momentum equations are advanced in time, the resulting velocity field generally does not satisfy mass conservation. To correct this, the Pressure Poisson Equation (PPE) is derived by taking the divergence of the momentum equation 2.1b and imposing the incompressibility condition 2.1a. Solving this equation is fundamental in enforcing the divergence-free velocity field required for incompressible flows [15]. However, this computation has proven to be computationally expensive with iterative methods such as Conjugate Gradient and Gauss-Seidel, which are inefficient due to their slow convergence at damping out low-frequency error components in the computational grid [15]. In some numerical procedures, solving multiple PPEs is required at each time step, making the computation very intensive. This significant cost necessitates highly efficient solution techniques.

Multigrid is a technique in which the conventional iterative procedure is combined with a correction of the solution on a sequence of coarser meshes. These methods accelerate the solution of linear (and non-linear) systems by applying a direct solving algorithm to the system on various levels of grid resolution [15]. Because solving algorithms (like Gauss-Seidel) are efficient at removing small-frequency wave lengths from the error distribution, but not at removing significant frequency errors, multigrid maps these larger frequencies to coarse meshes such that they appear smaller to the solver and will thus be more quickly resolved.

Multigrid is usually implemented over a grid hierarchy, where the algorithm cycles to progressively coarser grids to ensure faster convergence. The pattern the algorithm takes can take various forms, where the most often used paths are described by the V and W cycle [15]. Modern CFD solvers employ these multilevel multigrid methods in two distinct ways: geometric multigrid (GMG), where coarse levels are generated by reducing the resolution of the structured computational grid, and algebraic multigrid (AMG), where the hierarchy is constructed directly from the system matrix. AMG is particularly advantageous for unstructured meshes, where matrix connections lack a regular geometric pattern [15]. Using this recursive implementation, multigrid methods typically approach $\mathcal{O}(N)$ or $\mathcal{O}(N \log N)$ scaling, where N is the number of grid points [31]. Because of this property, multigrid methods are often described as having "optimal time complexity" [59].

This method has shown promising results for LES, where Gravemeier et al. [36] has demonstrated that a purely algebraic variational multigrid can improve scale separation in LES. This, in turn, led to more accurate simulations for a reduced computational cost.

Multigrid methods are one of the various methods that can be used to accelerate the convergence of ill-posed linear (or nonlinear) systems using an iterative solver. It is part of a collection of so-called pre-conditioning schemes, which are methods for making ill-conditioned or stiff matrix equations converge more quickly. More popular preconditioning schemes are the ones postulated by Turkel [105], Weiss and Smith [110], Meijerink and Vorst [65] and Chapman, Saad, and Wigton [16].

Alternative time stepping

Alternative time stepping is a method used in CFD solvers to improve convergence rates and stability by adjusting the time stepping scheme employed by the solver. Explicit time-stepping schemes, such as the forward Euler method, are particularly dependent on the Courant-Friedrichs-Lewy (CFL) number to maintain stability. The CFL number is influenced by the time step size chosen by the solver and is inversely related to the grid spacing. This means that in a constant time stepping scheme, the smallest grid cell dictates the system's stability.

To solve this issue, the time step can be adapted throughout the simulation. This is achieved by monitoring the solver's convergence and adjusting the time step to either ensure convergence or accelerate it. The most recent advances in Adaptive Time Stepping (ATS) have shown promising results when coupled with an implicit time integration scheme on false transient RANS schemes [73]. In false transient RANS, the time evolution is treated as a computational artefact, referred to as "pseudo-time," rather than a physically accurate temporal progression [73]. ATS has also been successfully applied to various turbulent flow cases, such as incompressible flow over a flat plate, the NACA-0012 airfoil, and various aircraft wings [73]. Where they have shown significant decreases in computational cost, using explicit time integration with adaptive time stepping is almost crucial; otherwise, computational time could be unnecessarily prolonged or become unstable.

Another way to increase the convergence rate by applying an alternative time-stepping scheme is to use local time stepping (LTS). LTS aims to minimise the global CFL restrictions by allowing different parts of the computational domain to use different time steps based on the local CFL requirement [28]. This approach is beneficial for simulations where certain regions require finer resolution than others, such as the inflow compared to the boundary layer of an airfoil. This situation typically creates meshes

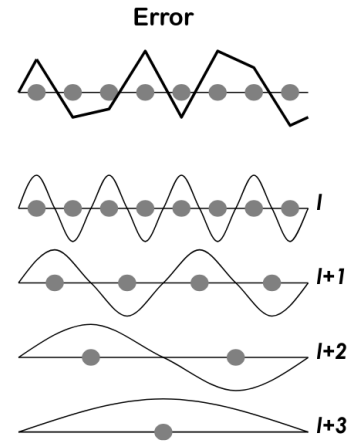


Figure 2.3: Multigrid error prolongation over various mesh spacings [66]

with a wide range of different cell sizes, and thus, the global CFL number is bounded by the smaller cells. LTS can generally be categorised into hierarchical and non-hierarchical approaches. Hierarchical methods employ techniques such as Adaptive Mesh Refinement (AMR), which will be discussed later in this document, to apply different time steps to different parts of the domain. Non-hierarchical methods focus on creating an interface between neighbouring grids that utilise different time steps [28].

Multigrid and alternative methods are just a few ways in which convergence can be stabilised or improved; numerous other methods achieve the same goal, which will not be listed in this literature review (see [73, 106, 28, 48, 102] and the references therein).

2.3.2. Mesh and model adaptations

Alongside alternative numerical and algorithmic techniques, simulations can also be accelerated by making mesh and/or model adaptations that can reduce computational costs. This subsection will discuss the various ways in which model adaptations and combinations have been accelerating CFD simulations.

Adaptive Mesh Refinement

Adaptive Mesh Refinement (AMR) is a framework that dynamically adjusts the mesh resolution of the computational domain based on the evolution of the solution. Initially developed for the numerical solution of hyperbolic partial differential equations Berger and Oliger [10] and later improved for conservation laws in Berger and Livermore [9], where AMR was used to efficiently resolve shockwaves in the solution, while limiting computational costs. AMR is beneficial for problems where physical length scales vary significantly across, or when complex features like shockwaves present themselves in the flow.

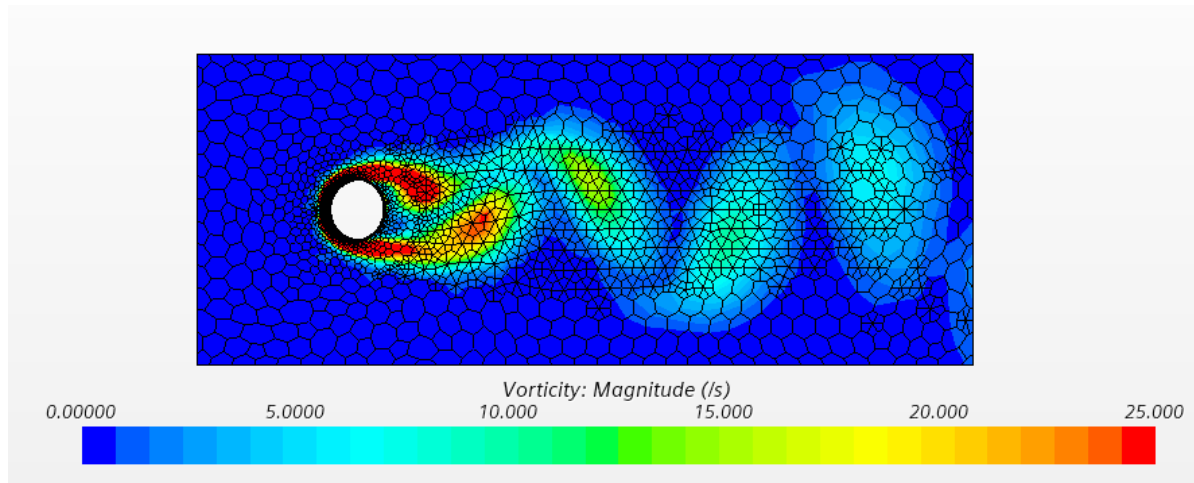


Figure 2.4: AMR being applied over the wake of a cylinder experiencing vortex shedding in its wake. Adapted from [61]

AMR employs a hierarchical grid system where the domain is first discretised with a coarse 'parent' mesh. Regions that require higher resolution are subsequently covered with a finer 'child mesh'. These finer child meshes will then also use a smaller time step than those of the parent mesh. This connects AMR to the ATS scheme explained previously. After this, the truncation error is assessed at each cell to determine where mesh refinement is needed [10]. If the truncation error of a specific cell exceeds the desired level, the grid is refined in this region. The algorithm then attempts to minimise overlap and limit the number of patches refined. On the other hand, cells which are no longer required to be refined can be dropped to save on computational cost. Figure 2.4 shows how AMR is being executed in the wake of the cylinder to capture the coherent structures being formed due to vortex shedding.

Applying AMR leads to a decrease in computational cost, as the dynamic control of the total number of cells minimises computing time while maximising solution accuracy by providing a highly resolved solution only where required. That is why AMR has become the method of choice, while in others, a technique of necessity, for fields like astrophysics and combustion [78]. One example of AMR is Shahsavan, Morovatiyan, and Mack [90], where AMR was used to investigate whether replacing nitrogen

with argon in the intake mixture enhances the auto-ignition and combustion characteristics of combustion engines. Because of its fullness, various frameworks have also been developed to make AMR available to a broader public [25, 62]

Wall models

Wall models are a crucial component for turbulent flow simulations, particularly for LES of wall-bounded flows. The computational cost of fully resolving the near-wall region is prohibitively expensive at high Reynolds numbers. Where the total work of determining the inner layer of the boundary layer scales with approximately $Re^{2.4}$ [52]. Wall models address this issue by modelling the effects of the innermost layer of the boundary layer on the resolved scales, which significantly reduces computational costs.

Wall models generally operate by replacing the traditional no-slip boundary condition at the wall by a modified stress boundary [77]. They can usually be categorised into two distinct groups: wall stress models and hybrid LES/RANS [52]. Wall stress models are based on the principle of energy conservation in a near parallel shear flow [52], meaning that the wall model aims to model the wall shear stress τ_{wall} by utilising the RANS boundary layer equation in Equation 2.6

$$\frac{\partial \bar{u}_i}{\partial t} + \bar{u}_j \frac{\partial \bar{u}_i}{\partial x_j} + \frac{1}{\rho} \frac{\partial p}{\partial x} = \frac{\partial}{\partial y} \left[(\nu + \nu_{t,wm}) \frac{\partial \bar{u}}{\partial y} \right] \quad (2.6)$$

The most common wall stress model is the equilibrium model, in which the convection and the pressure gradient balance each other out. This means that the left-hand side of Equation 2.6 is assumed to be zero. This leads to an ODE that, when using mixing length theory, has a solution in the form of $u^+ \approx y^+$ for $y^+ \lesssim 5$ and $u^+ \approx \frac{1}{\kappa} \ln(y^+) + B$ for $y^+ \gtrsim 30$, which is the model used for the algebraic wall stress model [52]. By implementing these solutions, the equilibrium model, and all other wall stress models, aim to match the velocity profile seen in 2.5

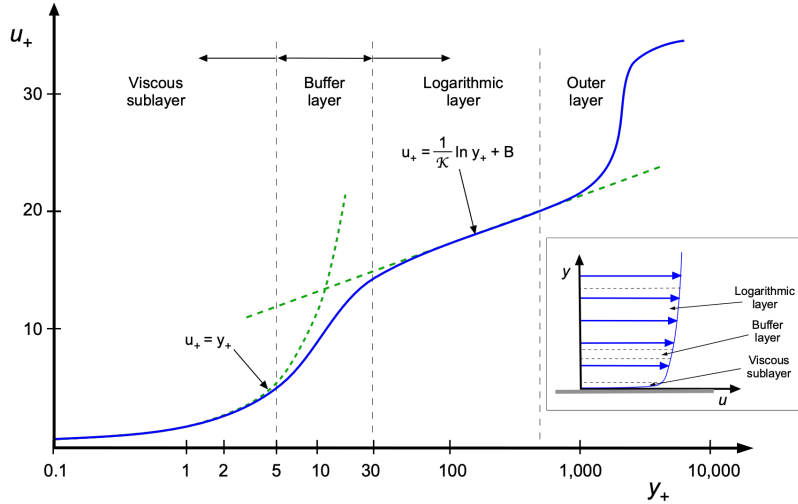


Figure 2.5: Velocity profile u_+ vs. y_+ showing the viscous sublayer, buffer layer, and log-law region. The linear and logarithmic approximations (dotted) are compared to the full profile (blue). Adapted from [56]

Another way to model the near-wall region is to resolve the left-hand side of Equation 2.6 locally, which leads to a more complex ν_t . This ODE approach [52] enables a more realistic wall model, but it does lead to higher computational costs. More advanced wall stress models have been proposed by Hickel et al. [40] and the references therein, which aim to incorporate non-equilibrium effects to predict flow separation better.

The other class of wall models are the hybrid RANS/LES models, where the idea is to apply a RANS approach to modelling the inner part of the turbulent boundary layer and an LES-type subgrid model to the outer layer cite Hickel2015. These can essentially be divided into zonal methods and blended methods. Where the former separate RANS and LES computations with a set interface, which can be

fixed by the user or dynamically adjusted by the solver, as was proposed in the work of Quéméré and Sagaut [80]. In contrast, blended methods switch between RANS and LES modes in a way that cannot be set by the user, with the most famous example being the Detached Eddy Simulation (DES). A good overview of these various wall models is presented in the work of Larsson et al. [52].

2.3.3. Parallelization techniques

Besides adaptations to meshing, time integration, numerical algorithms and modelling approaches, computational cost can be lowered through the strategic application of parallel computing architecture, utilising both CPU (Central Processing Unit) and GPU (Graphics Processing Unit) computation power. This approach is fundamental for handling the increasingly rising computational demands of complex turbulent flows and large-scale industrial or research problems [15]. Parallel computing or parallelisation is the simultaneous execution of multiple computational tasks on a multiprocessor system by distributing the computational load among the different processors [1]. By executing numerous tasks on various processing units, large tasks can be completed more quickly. In previous decades, performance gains in scientific computing were achieved through breakthroughs in hardware engineering, such as an increase in clock speed or lithographic advancements. However, as Shalf [91] indicated, the era of continuous performance gains has ended, partly due to limitations in clock speed scaling (due to excessive power consumption) and the increase in transistor density approaching physical limitations. This means that to ensure accelerations in the near future, architectural advances will be the most practical path to continue the growth of high fidelity CFD applications[91]. This subsection highlights key parallel computing strategies that have been developed over time.

OpenMP

Open Multi-Processing (OpenMP) is a widely adopted parallelisation tool which enables computations to be distributed across multiple threads running on the cores of a single processor or within a single shared memory system [1]. OpenMP's leading role in CFD is to enable parallel execution without requiring the decomposition of the computational domain or communication between processes, as all threads operate within the same memory space. Since the computational grid does not need to be subdivided into smaller subdomains, there is no need for communication between processes, which reduces the total computational cost of the algorithm and simplifies load balancing[1]. Due to its flexibility and ease of path development, OpenMP is supported by virtually all major compiler vendors. Making it a popular choice for enabling internal node parallelism.

The work of Afzal et al. [1] listed various examples of parallelisation of CFD codes using OpenMP. As his work is already quite extensive, this literature review will not provide examples of OpenMP implementation.

In the context of high-fidelity CFD applications, OpenMP does offer some disadvantages. These mainly arise from its limited scalability, due to its restriction to shared memory architectures, which means it struggles with enormous CFD problems that are difficult to load into a single memory cache [1]. For these significant CFD challenges, OpenMP is often deployed in a hybrid approach alongside tools like MPI, where it handles the parallel work on the cores within each node. This complements the role of MPI in coordinating computations across separate nodes. MPI and this hybrid model, along with their applications, will be explained further in this subsection.

MPI

Message Passing Interface (MPI) is also a key standard for parallelisation on distributed memory platforms. It provides a collection of commands which exchange messages and manage processes between processing units. Unlike OpenMP, which is designed for shared memory systems where all parts of the program can access the same data in a single memory pool, MPI is used for distributed memory systems, where each processing unit has its own private memory [33]. In the context of CFD, engineers often deal with vast computational domains that require extensive amounts of memory and processing power. Where using a single memory cache or processing unit would not be fast enough to solve these types of problems on its own. MPI allows the user to distribute these significant problems across many different nodes, where each node has its own private memory that the other nodes can't access. Each node (which can be a processor core or an entirely different computer) works on a portion of the overall simulation.

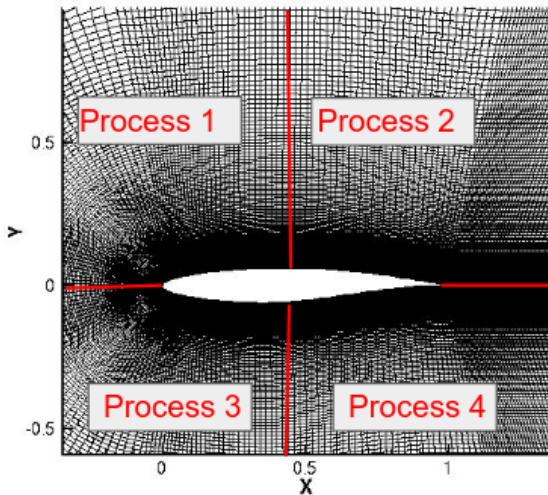


Figure 2.6: Decomposition of the computational domain around an airfoil across four different nodes [66]

To ensure the overall CFD solution is accurate and continuous, the sub-domains need to exchange information at their boundaries; MPI provides the communication protocol for this exchange. In CFD, data is exchanged for "ghost cells" or "halo cells" which are at the interface between neighbouring sub-domains. These cells contain data from adjacent blocks that a processor unit needs to correctly calculate its own part of the solution [1, 33]. An example of this mesh decomposition can be seen in Figure 2.6, where MPI ensures that processes 2 and 4 receive the correct flow information from processes 1 and 3, respectively.

Afzal et al. [1] reported various implementations of the MPI protocol, which achieved significant CFD speed-up results. These results generally follow the trends predicted by Amdahl's law [5], which, for CFD applications, limits the maximum theoretical speed-up to around 20x. However, Amdahl's law is only a theoretical bound; in practice, MPI implementations in CFD often achieve better scalability due to improved algorithms, communication strategies, and domain decomposition [33, 38].

CUDA

Due to advances in GPUs, the Compute Unified Device Architecture (CUDA) stands out as a prominent framework for leveraging GPUs to achieve significant computational speed-ups. CUDA, a platform developed by NVIDIA, provides a general-purpose parallel computing platform specifically designed for their GPU architectures [51]. When implemented, CUDA allows CFD solvers to employ the massive computational power and high memory bandwidth of the GPU, where it acts as a 'co-processor' to the CPU. The CPU, or "host," handles the sequential parts of the application and overall control logic, while the GPU, or "device," takes on the massively parallel computational tasks [113].

CUDA has enabled CFD researchers to achieve speedups of over 50% compared to the sequential CPU version, as demonstrated in the work of Lai et al. [51]. The authors benchmarked two commer-

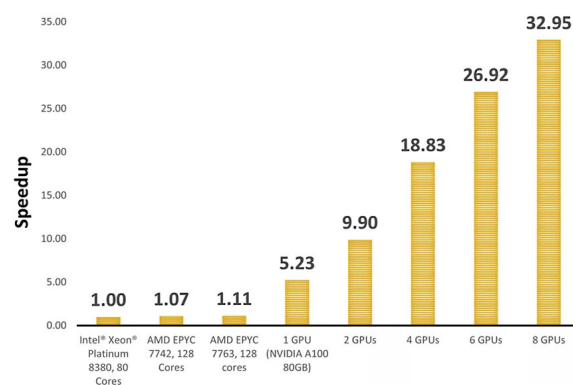


Figure 2.7: Speedup comparison of CFD simulations across CPU and GPU hardware configurations. The results show modest gains with CPU alone, while GPUs offer significant performance improvements. Adapted from ANSYS, Inc. [6]

cially available GPUs for the HSMAC (Highly Simplified Marker and Cell) and SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) algorithms in a 2D Lid-Driven Cavity Flow. GPUs allow a more cost-effective solution for parallel computing than expensive CPU clusters, making high-performance computing more accessible to researchers [17]. The work of Afzal et al. [1] illustrates more examples of CUDA parallelisation and computational speed improvements, which will not be discussed in this work as the primary focus lies on accelerating high-fidelity CFD cases. ANSYS, Inc. [6] has also demonstrated that one NVIDIA A100 GPU outperforms a HPC cluster with 400 Intel Xeon Platinum 8380 cores, while doing so for up to 7 times cheaper hardware costs.

Furthermore, GPU devices offer a more cost-effective and energy-efficient solution compared to expensive CPU clusters [1]. This ties back to the statement made earlier in this section, where clock speed advancements have to be limited due to high power demands.

While GPU acceleration through CUDA represents a powerful avenue for enhancing solver performance, especially for large-scale unsteady simulations, its integration often requires significant code restructuring and hardware-specific modifications [1]. Due to the large computational domain of high-fidelity CFD cases, such as LES, memory transfer between the CPU and GPU can present itself as a significant bottleneck in CUDA implementations. This is due to the considerably slow transfer speed over the CPU-GPU connection compared to the GPU's internal memory bandwidth [113, 1].

Hybrid parallelization methods

Knowing the strengths and weaknesses of OpenMP, MPI, and CUDA has motivated researchers to combine these methods into hybrid parallelisation frameworks, which aim to leverage the strengths of each of these methods. As mentioned before, MPI is well-suited for distributed memory systems and decomposition of large CFD fields, while OpenMP is utilised in finer parallelisation tasks. One example of OpenMP and MPI being combined is the work of Jin et al. [47], where MPI is used for communication across distributed memory nodes and OpenMP is used for fine-grained parallelisation within the node [47]. The work demonstrated that the combination of OpenMP and MPI yielded the best results when fewer MPI tasks were performed with moderate OpenMP threading.

OpenMP and CUDA have also been combined to leverage each other's strengths. The work of Yue et al. [114] leveraged these methods in a way that the parts that could not be parallelised over the GPU were parallelised by the CPU using OpenMP. Furthermore, optimisation of different types of memory was achieved by using shared memory to decrease latency between the GPU threads, thus promoting overall performance [1].

The combination of MPI and CUDA is a popular strategy for HPC clusters, combining the memory parallelism between nodes deliverable by MPI with the massive memory and computing power of GPUs. Similar to the OpenMP and CUDA hybrid method. The method implemented by Lai et al. [51] uses the CPU to manage the GPU execution and communication. At the same time, the GPU performs the primary data processing, which has shown significant improvements compared to CPU executions. Furthermore, the study by Jacobsen, Thibault, and Senocak [43] has also indicated the significant speed-ups that MPI-CUDA parallelisation can deliver in GPU HPC clusters, showing speed-ups of more than 100% for 128 GPUs [43].

Future Outlook and Current Bottlenecks

As previously mentioned, the future of using hardware to achieve accelerated CFD simulations is shaped by the end of the traditional processor scaling (Moore's law [68]) and an ever-growing need for more complex flow simulations, which is why companies like NVIDIA are enabling significant advancements in architectural specialisation. Shalf [91] predicts that a shift towards architectural specialisation is needed, as HPC platforms become increasingly heterogeneous. That is why one way of ensuring continued performance improvements is to use transistors more efficiently by specialising the architecture to the target scientific problem [91].

Due to the increasing complexity and size of CFD domains, data movement is presenting itself as a bottleneck in performance. Due to the vast amounts of data that need to be handled by the hardware architecture, the energy required to move data exceeds the energy used to process that data [91]. On the other hand, these vast amounts of data present a unique opportunity. The large amount of data generated from high-fidelity CFD cases can be leveraged to develop data-driven acceleration schemes

that approximate flow or surrogate models, replacing the computationally expensive parts of the solver. This is where machine learning and artificial intelligence techniques might find their place in the industry. These models can be trained on these large amounts of data to accelerate and can provide interesting predictions or optimised acceleration techniques. This way, the very data that seems to be a bottleneck to hardware acceleration schemes can enable interesting new CFD acceleration strategies, which will be explained in the rest of [section 2.4](#)

2.4. Machine/Deep learning Enhanced CFD

The integration of machine learning (ML) into the CFD field presents interesting opportunities for acceleration frameworks for turbulent flow simulations. Where traditional performance-enhancing methods in [section 2.3](#) can offer significant performance gains, they remain constrained by their reliance on numerical algorithms. This is where data-driven methods can complement these traditional methods by leveraging the massive amounts of available high-fidelity simulation data.

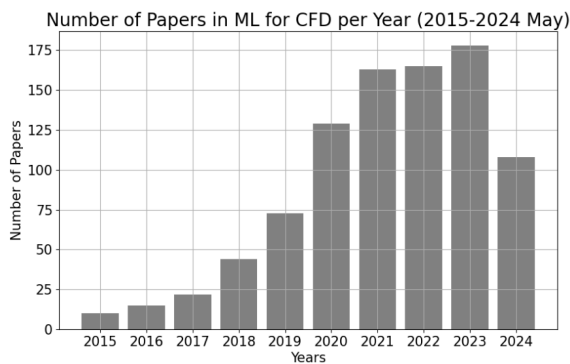


Figure 2.8: Rise in the amount of ML papers in CFD [109]

It is worth noting that recent years have seen an exponential growth in the number of studies exploring ML-enhanced CFD. Countless methods incorporating ML into the CFD landscape have been developed, ranging from broadly adopted to more niche applications. Comprehensive reviews, such as those by Vinuesa and Brunton [107] and Wang et al. [109], demonstrate the rapid advancements in the field. [Figure 2.8](#) visualises this trend, showing how the number of papers on ML for CFD has shown a steady increase in the last 5 years. Given the sheer volume of available material, it is beyond the scope of this work to cover all relevant contributions in detail.

Before highlighting the current landscape of ML-enhanced CFD simulations, [subsection 2.4.1](#) and [subsection 2.4.2](#) will briefly handle ML and NN terminology before [subsection 2.4.3](#) will introduce key neural network architectures which are frequently used in the CFD environment. Where [subsection 2.4.6](#), [subsection 2.4.5](#), and [2.4.4](#) will showcase how ML can be leveraged to accelerate CFD simulations by enhancing solver components, employing data-driven ROMS and creating physics-informed surrogate models.

2.4.1. Introduction to Machine Learning

Machine Learning is a branch of artificial intelligence that aims to develop algorithms capable of learning patterns and making predictions from data without being explicitly programmed. ML models can essentially be expressed as a function $y(x)$ which takes an input vector x and transforms it to an output vector y . The precise form of $y(x)$ is determined during the *training phase* based on the training data [11]. The training of the model requires a dataset that is usually split into training, validation and test sets. The training set is used to fit the model parameters, while the validation set can be used to determine hyperparameters, such as the learning rate and network architecture. The test set evaluates the model's generalisation to unseen data [13].

Depending on the target variable, ML objectives are typically categorised into two categories: regression (continuous outputs such as velocity or pressure predictions) and classifications (discrete categories, e.g., image recognition and Optical Character Recognition).

A central challenge in ML is ensuring that models generalise beyond the training data, meaning that they can reliably generate correct outputs for unseen data that was not in their training set. Overfitting occurs when a model has been trained to achieve an extremely high accuracy on the training data, resulting in poor predictions on unseen test data. While underfitting reflects a model that is too simple to capture the underlying dynamics of the training data, resulting in poor performance even on the training data [13]. To address these pitfalls, regularisation strategies, careful dataset design, and physically informed

constraints can be employed to reduce model complexity and penalise unrealistic predictions [2, 87].

ML algorithms are broadly grouped into three categories: supervised learning, where models learn from labeled data (most common in CFD, like mapping flow fields to forces); unsupervised learning, which identifies patterns or reduced representations from unlabelled data (e.g., clustering coherent flow structures or dimensionality reduction); and reinforcement learning, where an agent interacts with an environment to optimize a reward.

A detailed treatment of machine learning theory is beyond the scope of this work. Still, it can be found in standard references such as Bishop [11], Burkov [13], and Daniel [22], which provide the broader theoretical background underlying the CFD-oriented applications discussed further down in this work.

2.4.2. Introduction to Neural Networks

Neural networks (NNs) are a class of ML models inspired by the biological nervous system, designed to approximate complex nonlinear functions. They consist of interconnected computational units, or neurons, which receive inputs, process them through an activation function and produce an output. A general architecture of a NN can be seen in Figure 2.9. The use of non-linear activation functions enables NNs to act as universal function approximators [41], capable of representing highly complex mappings between inputs and outputs.

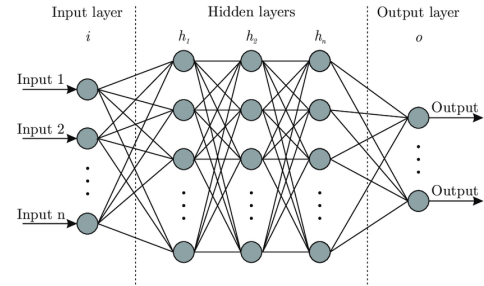


Figure 2.9: General architecture of a Feed Forward Neural network, adapted from [70]

Training a neural network involves minimising a loss function that quantifies the difference between predictions and actual values across the dataset. This optimisation is typically performed through gradient descent and its variants, with gradients computed efficiently using the backpropagation algorithm [2]. To achieve a desirable accuracy, hyperparameters such as learning rate, network depth, and activation function can be optimised by monitoring the network's performance. After training is completed, a set of weights and biases is created that defines how the network transforms inputs into outputs.

The architecture of a NN can significantly affect its performance on different tasks. The simplest form of NNs are feed-forward neural networks (FNNs), as seen in 2.9, where information flows from input to output in a single pass.

2.4.3. Common Neural Network Architectures Employed in CFD

Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a frequently used class of neural networks specifically designed to process grid-structured input data that exhibit strong spatial dependencies in local regions of the grid [2]. That is why CNNs are often used for image recognition, since adjacent spatial locations in an image often have similar colour values of the individual pixels. CNNs are characterised by convolution operations, which involve taking a dot product between a grid-structured set of weights (known as a filter or kernel) used to extract features from the input data.

Because of the strong capabilities of CNNs to extract features from image snapshots, they present themselves as good candidates to be implemented for CFD predictions. Since CFD flow fields can be represented as a structured grid, CNNs offer an outstanding possibility to learn patterns such as vortices or boundary layers effectively. This can be helpful for tasks such as flow prediction, surrogate modelling, or acceleration [55, 37].

Recurrent Neural Networks

Recurrent Neural Networks (RRNs) are specifically designed for sequential data, where elements are ordered and depend on the value of the previous component of the series. Unlike traditional FFNs, RRNs take the inherent order of the data into account by processing inputs in sequence and treating

them similarly relative to previous inputs. This behaviour is achieved by sharing the same set of network parameters across different temporal layers, allowing the network to maintain a hidden state that changes with each new input, effectively acting as a memory of past information [2].

However, RNNs often struggle to predict long-term dependencies due to the vanishing and exploding gradient problem, which can arise during training. To combat this, a variation of RNNs called Long Short-Term Memory (LSTM) networks addresses these issues by employing internal memory and gating mechanisms that regulate how information is stored, updated, or forgotten, allowing them to manage long-term dependencies more effectively. An overall structure of an LSTM is displayed in Figure 2.10, showing how the internal memory state c_t refers to the work by Aggarwal [2], which provides an in-depth theoretical analysis of LSTMs, which are considered to be outside the scope of this literature review. For a more detailed explanation of RNNs, LSTMs, and the vanishing and exploding gradient problem, please refer to the work by Aggarwal [2], which provides an in-depth theoretical analysis of these concepts.

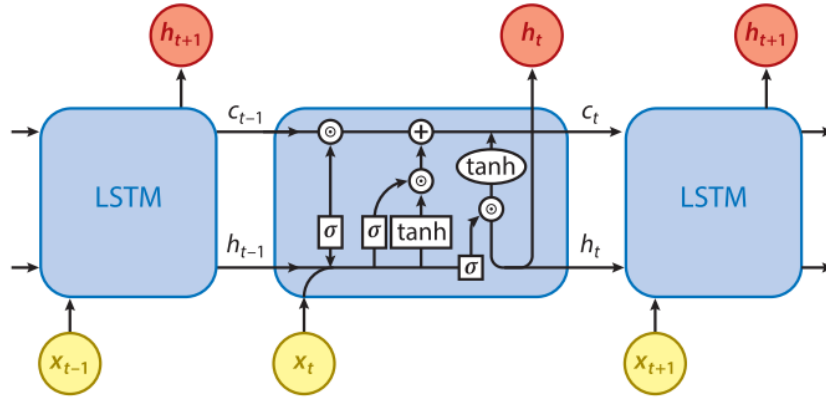


Figure 2.10: Structure of an LSTM cell with input, forget, and output gates regulating information flow through the memory cell. Source [12]

LSTMs are particularly useful in CFD due to fluid dynamics being an inherently transient environment where time-dependent phenomena play a key role in accurate simulations. LSTMs generally excel at predicting temporal sequences and capturing complex non-linear dynamics over longer durations [2]. These capabilities have been demonstrated to be beneficial in enhancing CFD simulations and will be showcased later in this work.

Neural Ordinary Differential Equations

Neural Ordinary Differential Equations (NODEs) are a type of deep neural network model introduced by Chen et al. [18] that differ substantially from conventional neural networks. They are similar to RNNs, and more specifically, Residual Neural Networks (ResNETs [39]). In a ResNet, the latent space is represented by a stack of layers, as shown in Figure 2.11. NODEs learn the mapping of sequential data through a continuous representation of the latent space.

Where the stacking of the latent state in ResNets can be seen as an Euler discretisation of a continuous transformation, shown in Equation 2.7

$$h_{t+1} = h_t + f(h_t, \theta_t) \quad (2.7)$$

Where h_{t+1} is the newly determined latent state, which is constructed of a summation of the current latent state h_t through the addition of a function of the network parameters and value of the current prediction $f(h_t, \theta_t)$, when the added layers become large and the

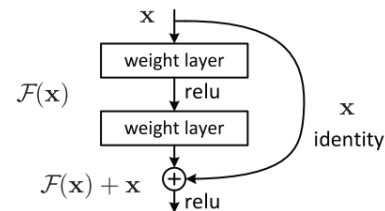


Figure 2.11: ResNet layer stacking, a building block.

Figure 2.11: ResNet layer stacking, adapted from [39]

steps taken become smaller, this progression of the latent state can be represented by a differential equation 2.8.

$$dh(t)/dt = f(h(t), t, \theta) \quad (2.8)$$

A conventional ODE solver can then be used to solve this equation. This formulation provides adaptive computation capabilities, as ODE solvers can dynamically adjust step sizes based on the solution's complexity, enabling an optimal trade-off between speed and accuracy. This essentially allows NODEs to define a smooth flow through the latent space, instead of a discrete series of transformations that ResNets produce. Figure 2.12 shows a graphical representation of the latent space vector field.

NODEs also provide a memory-efficient way to train deep models using the adjoint sensitivity method, which propagates through the ODE by solving a second augmented ODE backwards in time. Because the neural network can be viewed as a universal approximator, and the ODE solver can evaluate the latent layers at any time point for any timestep, the resulting model provides a smooth prediction without being constrained to discrete time steps or layers. This behaviour makes NODEs particularly well-suited for predicting systems where continuous time behaviour is essential, even if the supplied training data is irregularly sampled in time.

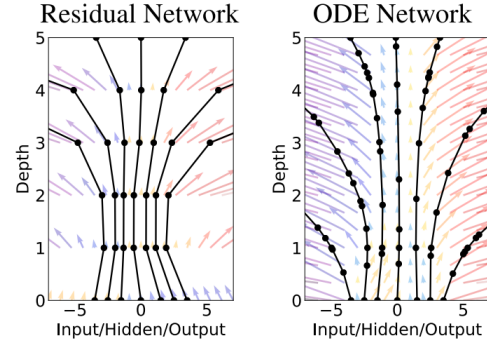


Figure 2.12: Left: Discrete evolution of ResNet predictions, Right: continuous evolution of predictions by NODE. Adapted from [18]

In the context of CFD time-series prediction, NODEs can offer key advantages. Their adaptive behaviour can allow for variable time step integration, which can be optimal in accelerating potential CFD simulations.

However, since NODEs involve solving ODEs and fluid systems are often high-dimensional, solving these ODEs can be computationally intensive. That is why, in recent literature, such as the work of Shankar et al. [92], NODEs have been implemented within data-driven reduced-order models, where the dynamics of the latent space are predicted using NODEs. This aligns with the objective of this thesis, where instead of predicting the time integration of the NSE, the authors have implemented NODEs in the latent space of a ROM, which is used to predict the entirety of the latent dynamics, which can then be used as a surrogate model for a specific flow case.

Another recent advancement in Neural ODE research has been the implementation of a physics-informed loss term. This embeds the governing equations of the system that the model aims to predict directly into the training process. Sholokhov et al. [93] introduced a collocation-based physics-informed loss function that enforces PDE constraints at sampled points, improving extrapolation, stability, and performance in situations where there is not a lot of training data available. Additionally, Ghanem et al. [32] proposed an optimisation framework that incorporates physics-informed loss terms, enabling the accurate recovery of hidden states when only partial measurements are available as training data. These works demonstrate that embedding physical laws into the NODE loss function allows for a more robust model in scenarios with sparse, noisy, or incomplete data. This type of approach could be particularly valuable in CFD, where enforcing conservation laws and physical consistency can lead to better flow field predictions using fewer training data.

2.4.4. Machine Learning Enhanced Solver Components

One way of leveraging ML to accelerate scale-resolved CFD simulations is by embedding machine learning directly into specific components of CFD solvers. Rather than learning the whole flow evolution, these approaches aim to improve the efficiency and accuracy of key solver stages. Calculations such as the Pressure-Poisson equation, RANS turbulence closure models, sub-grid scale models, or even learning-based discretisation are all possibilities that have been presented in recent literature.

Pressure-Poisson prediction

As mentioned previously, an essential component of incompressible CFD solvers is the pressure-velocity coupling algorithm, which ensures that the computed velocity field remains divergence-free. This coupling is typically enforced in operator splitting methods to discretise the NSE, where first the velocity field is advected, and the resulting velocity field $\mathbf{u}^*$ does not satisfy the continuity equation 2.1a. This is then corrected using the Pressure-Poisson Equation (PPE) 2.9 [107].

$$\frac{\Delta t}{\rho} \nabla^2 p = -\nabla \cdot \mathbf{u}^* \quad (2.9)$$

Where Δt is the simulation time step, ρ the fluid density and p the pressure field, finding the solution to this equation is typically the most computationally expensive for the numerical solver. This is why exploring alternative strategies for this computation could present promising acceleration results.

Chen, Jin, and Li [19] proposed a framework which employed a multi-scale Graph Neural Network (GNN) to either entirely or partially replace the traditional iterative PPE solver. Using this framework, the authors achieved solution speeds nearly three times faster for a Reynolds number of 1000 and almost twice as fast for 5000 in the training datasets. Additionally, Ajuria et al. [3] developed a hybrid computational strategy to accelerate the PPE computation. The authors propose a method using Convolutional Neural Networks (CNNs) to predict the pressure field in addition to the traditional Jacobi solver. Using this hybrid method, the authors achieved significant speed-ups of up to 320% while maintaining the predetermined accuracy. The authors, however, do not mention how long it took to generate the training data and how long the CNN training took.

Recently, Sousa, Afonso, and Rodrigues [96] proposed a versatile ML-based method applicable to various geometries. The developed ML models utilise Multilayer Perceptron (MLP) neural networks combined with Principal Component Analysis (PCA) transformations. This combination has proven to be efficient and trainable, requiring relatively low computational resources, and achieves acceptable accuracy with errors of around 3%. However, the authors note that the surrogate models struggle with extrapolating into significantly different flow regimes. To address this limitation, they suggest diversifying the dataset to enhance the models' practical applications.

Turbulence modelling

In the context of RANS simulations, the modelling of the RST is essential for providing closure to the Reynolds-averaged simulation 2.4. Traditional closure models, such as eddy viscosity models, often rely on empirical formulations which may not generalise well across different flow conditions. To address these limitations, ML frameworks are being proposed to offer a promising approach for developing data-driven turbulence models that can learn more accurate and applicable representations of the RST from high-fidelity data (like DNS or experimental results).

One of the most notable models is the one by Ling, Kurzawski, and Templeton [58], which leverages a custom-built Deep NN architecture which forces the network to make physically accurate predictions. This was achieved by embedding Galilean invariance into the predicted tensor, which improved the network's performance, enabling it to outperform traditional eddy viscosity models [107]. More recently, Cai et al. [14] proposed an improved version of the Tensor Basis Neural Network (TBNN), which builds upon the work of Ling, Kurzawski, and Templeton [58]. Their proposed framework used MLPs and a loss function which employed a weighted MSE based on the Reynolds stress anisotropy tensor components. Using this approach, they achieved a global R^2 improvement from approximately 0.7 to 0.9982 for Square Duct Flow (SDF). They significantly improved the prediction of the components of the Reynolds stress anisotropy tensors from 0.3295 to 0.9899 for specific entries. Furthermore, transfer learning was also implemented to circumvent data scarcity, which improved performance and delivered faster convergence with only 10% of the training data. Additional frameworks like

Although recent models, such as the aTBNN [14], and more recent works, like [34, 46, 88], have shown clear improvements over conventional models, ML-based turbulence modelling is still not ready for widespread use in engineering practice. The models often work well on the flows they are trained on, but struggle to generalise to new or complex cases. Recently, the NASA Symposium on Turbulence Modelling [86], along with a self-released paper by Spalart [99], highlighted that there are still

fundamental issues in the RANS framework, and machine learning alone will not resolve them. The limited (but increasing) amount of high-fidelity data available, combined with the lack of robustness in industrial settings, makes it challenging for engineers to trust these models fully. As a result, despite the interesting progress, the field still faces growing pains and will need collective transparency and standardisation before it can be used reliably in real-world applications.

Sub grid scale modelling

Similar to the closure problem in RANS modelling, LES needs to model the effect of the sub-grid scales on the resolved scales to ensure accurate simulations. Conventional SGS models typically rely on case-specific constants, such as scale similarity [7] or eddy viscosity [57, 30], which may not be applicable across all flow regimes and can lead to significant errors if not correctly configured. With the increasing availability of high-fidelity data and improved computational resources [6, 17], ML can offer alternative SGS models that are more adaptive, providing the potential to capture the complex interactions between turbulent scales.

ML has been used to develop SGS models in basically two ways: supplementing the unresolved energy in the coarse mesh using supervised learning and developing reinforcement learning approaches to stabilise the coarse simulation [107]. For the first approach, numerous studies exist that employ high-fidelity data to train deep learning models. The goal is to train these models to map the energy of the small-scale structures of the fine grid DNS data to the coarse LES mesh. The first one to propose such a model was Beck, Flad, and Munz [8], who employed CNNs to determine the underlying unknown non-linear mapping from the coarse grid quantities to the closure terms without a priori assumptions [8]. They achieved a cross-correlation of 47% in decaying homogeneous isotropic turbulence. Alternatively, Sirignano and MacArt [94] developed ML closure models for LES of incompressible flows around bluff bodies. Unlike a priori methods that train neural networks before implementing them into the CFD solver, they proposed an a posteriori approach, which directly optimises the neural network parameters by solving the LES equations and their adjoints. Their ML-LES models significantly outperformed commonly used dynamic Smagorinsky and Anisotropic Minimum-Dissipation (AMD) [85] models in a posteriori predictions, even on relatively coarse meshes. Even when training the model on rectangular bluff bodies, their framework showed promising results for out-of-sample bluff body shapes. This indicates that ML frameworks for LES closure models can be generalised to alternative flow cases if their architecture and training samples are adapted accordingly.

An example of reinforcement learning being employed for developing a data-driven SGS model is the work by Kurz, Offenhäuser, and Beck [50]. This method addresses the inconsistency of supervised learning for implicitly filtered LES, where the exact form of the implicit filter is typically unknown, making it impossible to compute corresponding closure terms a priori. By formulating turbulence modelling as an RL task, the agent, based on a CNN-based policy, interacts directly with the dynamical LES environment to dynamically adapt the eddy viscosity in space and time, based solely on the local flow state. Starting from an initial flow state, the agent will keep interacting with the environment until a desirable state is reached, an overview of this process is shown in Figure 2.13. The research demonstrates that the trained RL-based models achieve long-term stable simulations and significantly outperform established analytical models in accuracy, while also exhibiting strong generalizability to different resolutions and higher Reynolds numbers not seen during training.

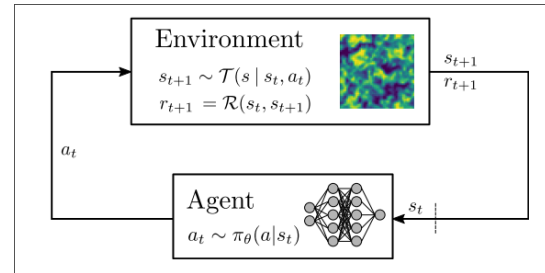


Figure 2.13: The agent interacts with the LES environment to adapt the eddy viscosity dynamically based on the local flow state, adapted from [50]

The examples highlighted are just a few of many ML-SGS models proposed; the work by Vinuesa and Brunton [107] offers a more detailed overview of alternative proposals for the same closure problem. However, similar to the RANS closure problem, these ML-SGS still suffer from poor generalizability. This is mainly due to the limited DNS data, which is often only available at low Reynolds numbers [87].

2.4.5. Data-Driven Reduced Order Models

Data-driven reduced-order models have been utilised to accelerate CFD simulations by approximating high-fidelity systems in a lower-dimensional form that retains most of the relevant flow phenomena. Data-driven ROMs extract dominant patterns from simulation or experimental snapshots without requiring explicit knowledge of the appropriate governing equations. Unlike physics-based ROMs, which typically use linear projections (such as POD) to solve the NSE on a basis with a reduced number of DOFs, data-driven ROMs rely on snapshots of the full-dimensional state vector to produce projection operations through a training process. This utilises the previously explained capabilities of neural networks to approximate a nonlinear function. These methods can achieve significant speedups after being successfully trained on supplied CFD snapshot data, or they can be used in a transfer learning fashion to accelerate CFD simulations, as will be explained in further detail.

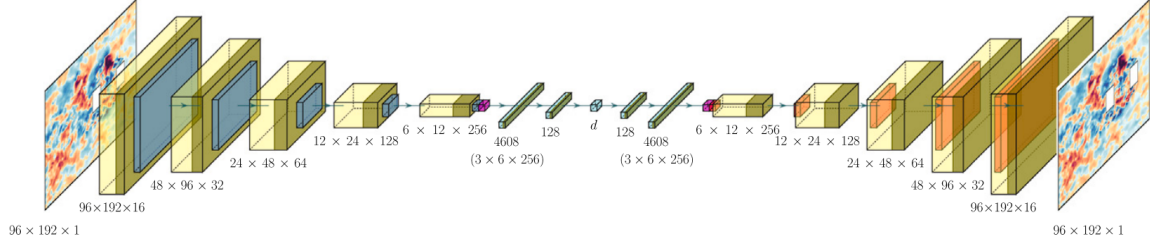


Figure 2.14: Schematic view of a CNN autoencoder extracting non-linear modes from a CFD snapshot, source [27]

Data-driven ROMs employ the same working principles as autoencoders, specifically the use of the encoder and decoder network architectures. The encoder compresses the high-dimensional snapshots into a lower-dimensional latent space, and the decoder reconstructs this latent space back to the exact dimensions as the input snapshots, while minimising reconstruction error. The usage of non-linear activation functions within the autoencoder allows the network to capture the complex non-linear behaviour of turbulent structures. This offers a key advantage over methods like POD, which are limited to linear representations [75]. For processing spatial data of fluid flows, autoencoders often integrate CNNs into their architecture due to their good results with grid-structured data. Figure 2.14 shows an overview of how an autoencoder reduces and reconstructs a single CFD snapshot to and from a latent space.

What differs data-driven ROMs from autoencoders is the requirement that the latent space must also capture and predict the temporal dynamics of the system. This means that, in addition to learning a compact representation of the input data, the latent variables must be predicted in a way that accurately reproduces the future state of the simulation [107]. This can be achieved by either selecting the dimensions of the latent space in a way that allows for a correct prediction of the future flow field, or by utilising regression architectures that are better suited for time series prediction, such as RNNs, LSTMs, or neural ODEs.

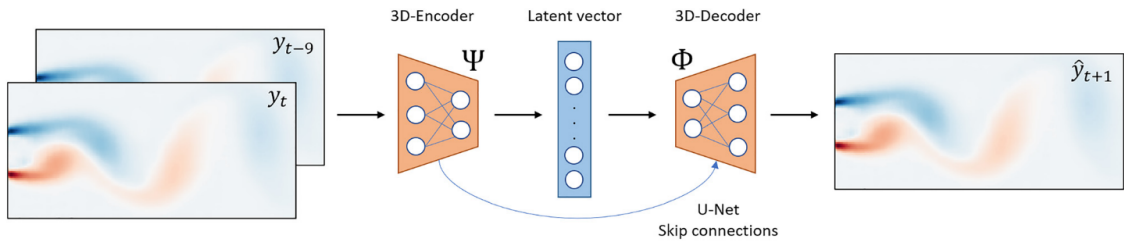


Figure 2.15: Example architecture of deep convolutional ROM, source [75]

Some notable examples of deep learning-based ROMs include the work of Pant et al. [75], where the authors leveraged 3D Autoencoder and 3D U-Net-based architectures to create non-linear projections from reduced-order states to fluid flow snapshots, as depicted in Figure 2.15. The authors employ a looped prediction strategy, where the last 10 snapshots of a simulation are used to train the network to predict the following timestep $\hat{y}_{t+1}$. This prediction is then appended to the last nine snapshots

and used as input to predict $\hat{y}_{t+2}$. This process is repeated recursively for 20 timesteps. This looping strategy allowed a reduction in simulation speed of 2 orders of magnitude [75]. Similarly, CFDnet [74] accelerates RANS simulations by leveraging its CNN ROM to predict fluid properties, while still utilising the CFD solver to ensure that domain-specific convergence constraints are met. This allowed the authors to deliver highly accurate predictions while still achieving acceleration rates from 1.9 up to 7.7 times the benchmark simulations.

As previously mentioned, the dynamics in the latent space can also be modelled by utilising a regressor in the latent space of the autoencoder, which would make the architecture of the data-driven ROM look like Figure 2.16. A notable example of such an architecture is the work by Shankar et al. [92], which demonstrates how NODEs are utilised to model system dynamics in the latent space. By explicitly decoupling the reduction and latent system dynamics, the presented framework achieved very accurate reconstructions of flow dynamics over large integral timescales. Instead of using NODEs for the dynamical modelling in the latent space, LSTMs can also be used, as they present good capabilities for time series predictions, as indicated in the work by Mohan et al. [67].

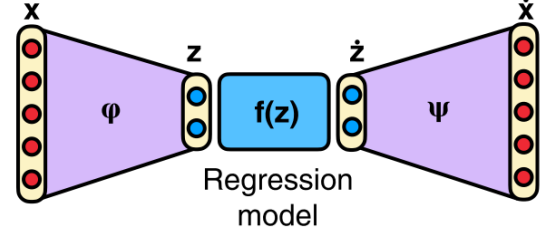


Figure 2.16: Example architecture of autoencoder with regression in the latent space, adapted from [107]

2.4.6. Physics Informed Surrogate Models

Physics-informed surrogate models demonstrate strong potential in accelerating computationally intensive simulations. A surrogate model is a broad term for a computationally efficient approximation of a complex system. A ROM, which was handled previously, is a specific type of surrogate that reduces the system's dimensionality by applying a projection to the high-dimensional dynamics in a lower-dimensional space. Meaning that all ROMs are essentially surrogate models, not all surrogates are ROMs.

Where purely data-driven surrogate models, such as DeepONet [60], require a large amount of data for reliable prediction performance. In real-world scenarios, where data is either sparse or prohibitively expensive to collect, models that can generalise effectively on smaller datasets are necessary [24]. That is why physics-informed surrogate models embed fundamental physical laws and constraints directly into their architecture or training process. This integration aims to create efficient approximations that are not only accurate but also physically consistent, even when dealing with limited training data [81].

The integration of physics into these models is achieved mainly through two key mechanisms: physics-informed loss functions and specialised network architecture. Physics-informed loss functions incorporate the residuals of the governing partial differential equations, such as conservation of mass 2.1a and momentum 2.1b, into the loss function during training. This way, the model can be regularised during training, showing bigger errors for predictions which violate the relevant physical laws. Essentially, an artificial bias is added to the network, which promotes data efficiency since the projections are inherently directed to adhere to physical laws. Equation 2.10 shows a general form of a physics-informed loss function, where $\mathcal{L}_{\text{data}}$ is a standard loss function like MSE or MAE, and $\mathcal{L}_{\text{physics}}$ can be derived from the physical system the network has to learn. In the case of a CFD simulation, this latter part can be built from the continuity and momentum equations. These two loss functions can then be combined using a weighted sum, with λ_d and λ_p serving as weights.

$$\mathcal{L} = \lambda_d \mathcal{L}_{\text{data}} + \lambda_p \mathcal{L}_{\text{physics}} \quad (2.10)$$

A prominent example is Physics Informed Neural Networks (PINNs), first proposed by Raissi, Perdikaris, and Karniadakis [81], that use deep learning to solve PDEs, exploiting the concept of automatic differentiation used in the back propagation algorithm to calculate partial derivatives [107]. PINNs have been utilised in various CFD scenarios, including hydrodynamic surrogate models [24], high-speed flow approximations [63], and more general NS cases [26, 103], where they have demonstrated excellent results in predicting flow parameters with limited training data. However, in the context of accelerating

fluid simulations, PINNs have a disadvantage in that their training speed restricts them from achieving significant speed-up results in a CFD workflow [45, 35]. On the other hand, PINNs have been successfully used as closure models for RANS and LES frameworks; the workings of this will be explained later in this literature review.

Alternatively to PINNs, the Finite Volume Method Network FVMN [44] is a physics-informed surrogate model which specifically exemplifies a physics-informed surrogate model tailored for CFD, integrating principles of the finite volume method (FVM) directly into its design. Unlike previously explained methods that only include information about a single grid point, FVMN employs a tiered input system that incorporates physical quantities from a central grid point and its neighbouring grid points into the input matrix of the neural network. This allows the network to imitate the mass and heat transport among neighbouring cells, which is considered the most crucial process in the FVMN. Additionally, instead of directly predicting the future state, FVMN employs a derivative output system, which means it predicts the change or derivative of physical quantities over a given time step.

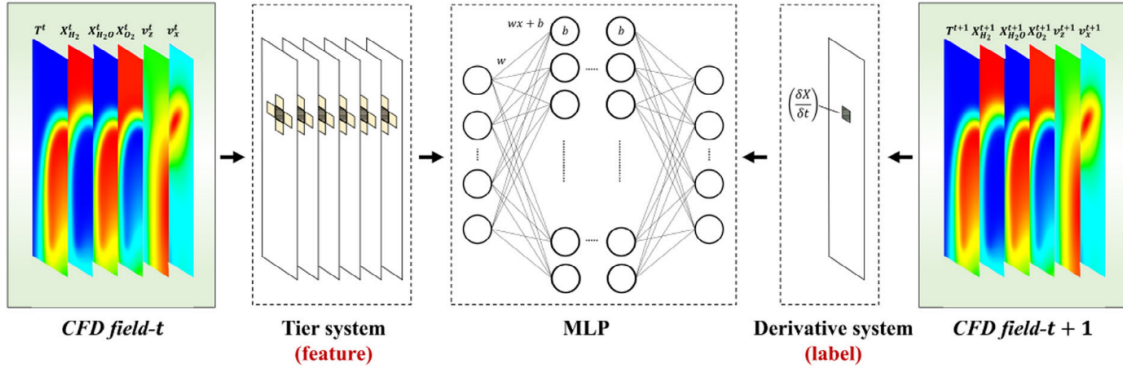


Figure 2.17: Overview of the FVMN architecture, showing the tier-derivative input/output system built around a conventional FFN (or MLP), adapted from [44]

In the original work Jeon, Lee, and Kim [44], significant improvements in accuracy were showcased compared to general models without the tier/derivative system implemented, where the authors managed to achieve a relative error with respect to CFD data of not more than 0.05%, compared to 1.12% for the conventional model.

In the context of accelerating CFD simulations, the FVMN also showed promising results. Due to the system's linearly increasing gradient error when used for longer transient predictions, the authors proposed an ML-CFD cross-coupling framework. The Residual-based Physics Informed Transfer learning strategy (RePIT) [45] is a hybrid framework where both a CFD solver and the FVMN alternately predict time series of fluid flows. RePIT continuously monitors the residuals of the governing equations, NSE 2.1, which are part of the physics-informed loss function employed by the FVMN in the case of fluid flows. Figure 2.18 shows a schematic overview of the transfer learning framework, it shows that when the residuals of the loss function (ε) exceed a predefined tolerance (ε_{th}), the CFD solver will use the latest prediction and continue solving the governing equations to reduce the residuals back to a desired value. Afterwards, these newly computed CFD snapshots are used to update the network parameters, allowing for new predictions to be made.

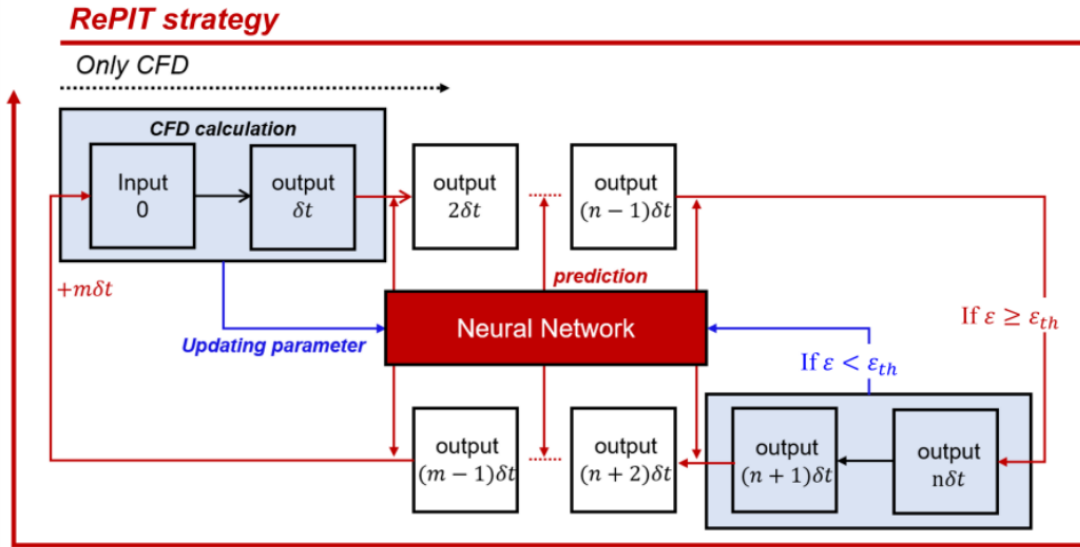


Figure 2.18: Schematic overview of the RePIT strategy showing the transfer learning process, adapted from [45]

This framework leads to significant computational speed-ups without compromising accuracy. The authors implemented this method in a natural convection CFD case, achieving a 1.9 times acceleration, including network parameter updates. This demonstrates the strong capabilities of the FVMN, as its tier-based input system requires a minimal amount of training data. RePIT shows strong potential as it is universally adaptable to various flow cases and boundary conditions.

While RePIT and FVMNs approximate spatial derivatives using mesh-based network structures, they can achieve smoother predictions by enabling the functionality of a continuous integrator, such as NODEs. Embedding NODEs into the RePIT framework would merge its residual-based transfer learning with NODEs' ability to handle adaptive time integration and irregular sampling. This combination has the potential to enhance stability, improve generalisation across flow regimes, and further reduce the computational cost of turbulent CFD simulations

2.4.7. Challenges and Future Directions

Machine learning has presented interesting solution paradigms for accelerating CFD simulations across various fronts. The most promising direction is the development of hybrid ML-CFD methods. Where traditional CFD solvers and ML models alternate between calculating/predicting time series data, methods like RePIT [45] and CFDNet [74] demonstrate that, when training data and loss functions are controlled, error accumulation can be mitigated, yielding significant speedups. Their online operation also enables the transferability of hybrid methods to a range of flow cases, making them more applicable to real-world CFD workflows.

The ML-enhanced CFD field is also showing promising advances through more sophisticated architectures and loss functions. Frameworks such as PINNs [81] and PNODEs [93] can enforce physical constraints, such as incompressibility, and learn from observational data to provide accurate predictions of complex flow fields. It can therefore be argued that the embedding of physical laws into neural network architecture is an essential feature for future acceleration frameworks.

Despite these advancements, key challenges and opportunities remain in the field. One of these is the interpretability of the ML models. Due to their often complex nature, ML models tend to be a black box model. This opaqueness hinders their performance as effectively as physics-based models, making it difficult to understand why a model made a specific prediction [107, 34]. This is a critical issue for generalisation, as models trained solely on data may struggle with unseen geometries and flow conditions, or when extrapolating beyond the training data's probability distribution. While advancements in hardware architecture [6, 17] and automatic data generation continue to aid in providing the high-quality datasets needed for these models, the ultimate goal is to enable ML to not only accelerate simulations but also advance discoveries of new physical mechanisms and revisit empirical laws in fluid mechanics through more transparent and quantifiable models.

While efforts continue for automatic data generation and automated experimentation pipelines to provide the large, high-quality datasets needed, the ultimate goal is to enable ML to not only accelerate simulations but also to assist in discovering new physical mechanisms and revisiting existing empirical laws in fluid mechanics, by yielding more transparent and certifiable models [12, 107].

Despite the numerous advancements made in applying machine learning to CFD, the current literature still reveals several limitations. Many studies have employed NODEs only within reduced-order models [84, 92], rather than integrating them directly into the CFD solver to accelerate the core time-integration step. Notably, their ability to serve as continuous time integrators has not been fully leveraged in scale-resolved simulations. That is why this work proposes applying a similar method to the RePIT framework by replacing its FVMN with NODEs. The RePIT framework showed that long-term stable acceleration can be achieved by alternating between CFD and machine learning predictions, guided by residual monitoring of the governing equations. The implementation of NODEs into the RePIT strategy can thus leverage RePIT's residual-based transfer learning, complemented by NODEs' continuous-time formulation. This strategy aims to directly address the bottleneck of time integration in CFD solvers, while ensuring that physical consistency is maintained through the use of residual monitoring. By pursuing this integration, the framework seeks to provide a concrete answer to the research questions introduced earlier: whether NODEs can deliver solver-level acceleration without compromising accuracy, and how residual monitoring can make such integration feasible.

3

Methodology

This chapter describes the full methodology used to develop and evaluate a machine-learning-based acceleration framework for CFD simulations. The approach combines a convolutional autoencoder (AE) with a neural ordinary differential equation (Neural ODE) to learn and evolve low-dimensional representations of turbulent flow fields, enabling significant speed-ups over conventional time integration.

3.1. Overall Pipeline

3.2. CFD Simulation Setup

3.2.1. Physical Setup

3.2.2. Numerical Solver

3.2.3. Dataset Construction

3.3. Data Preprocessing

3.3.1. Ghost Cell Removal

3.3.2. Channel Construction

3.3.3. Normalization

3.4. Autoencoder

3.4.1. Architecture

3.4.2. Loss Function

3.4.3. Training Setup

3.4.4. Validation

3.5. Neural ODE

3.5.1. Architecture

3.5.2. ODE Solver

3.5.3. Training with Multiple Shooting

3.5.4. Validation

3.6. Out-of-Distribution Detection

3.6.1. KNN-Based OOD Scoring

3.6.2. Fallback Strategy

3.7. Integrated Acceleration Workflow

3.7.1. Fixed-Horizon Prediction

3.7.2. Flexible-Horizon Prediction

3.7.3. Coupling with the CFD Solver

3.8. Evaluation Design

3.8.1. Accuracy Metrics

3.8.2. Speedup Metrics

3.8.3. Generalization Tests

3.8.4. Ablation Studies

3.9. Implementation Details

3.9.1. Software Stack

3.9.2. Hardware and HPC Configuration

3.9.3. Reproducibility

References

- [1] Asif Afzal et al. "Parallelization Strategies for Computational Fluid Dynamics Software: State of the Art Review". In: *Archives of Computational Methods in Engineering* 24 (2 Apr. 2017), pp. 337–363. ISSN: 18861784. DOI: [10.1007/s11831-016-9165-4](https://doi.org/10.1007/s11831-016-9165-4).
- [2] Charu C. Aggarwal. "An Introduction to Neural Networks". In: *Neural Networks and Deep Learning: A Textbook*. Cham: Springer International Publishing, 2018. ISBN: 978-3-319-94463-0. DOI: [10.1007/978-3-319-94463-0_1](https://doi.org/10.1007/978-3-319-94463-0_1). URL: https://doi.org/10.1007/978-3-319-94463-0_1.
- [3] E. Ajuria et al. "Towards a hybrid computational strategy based on deep learning for incompressible flows". In: *AIAA AVIATION 2020 FORUM*. Vol. 1 PartF. American Institute of Aeronautics and Astronautics Inc, AIAA, 2020, pp. 1–17. ISBN: 9781624105982. DOI: [10.2514/6.2020-3058](https://doi.org/10.2514/6.2020-3058).
- [4] Altair Engineering Inc. *Reynolds–Averaged Navier–Stokes (RANS) Simulations*. https://2021.help.altair.com/2021/hwsolvers/acusolve/topics/acusolve/training_manual/rans_simulations_r.htm. AcuSolve Training Manual. Accessed: 2025-06-10. 2021.
- [5] Gene M. Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities". In: *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*. AFIPS '67 (Spring). Atlantic City, New Jersey: Association for Computing Machinery, 1967, pp. 483–485. ISBN: 9781450378956. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560). URL: <https://doi.org/10.1145/1465482.1465560>.
- [6] ANSYS, Inc. *Unleashing the Power of Multiple GPUs for CFD Simulations*. Accessed: 2025-06-17. ANSYS, Inc. 2023. URL: <https://www.ansys.com/blog/unleashing-the-power-of-multiple-gpus-for-cfd-simulations>.
- [7] Jorge Bardina, Joel H. Ferziger, and William C. Reynolds. "Improved Subgrid-Scale Models for Large-Eddy Simulation". In: *AIAA Journal* 18.4 (1980). Seminal work proposing the scale-similarity SGS model for LES., pp. 1061–1068. DOI: [10.2514/3.50819](https://doi.org/10.2514/3.50819). URL: <https://doi.org/10.2514/3.50819>.
- [8] Andrea Beck, David Flad, and Claus Dieter Munz. "Deep neural networks for data-driven LES closure models". In: *Journal of Computational Physics* 398 (Dec. 2019). ISSN: 10902716. DOI: [10.1016/j.jcp.2019.108910](https://doi.org/10.1016/j.jcp.2019.108910).
- [9] M J Berger and Lawrence Livermore. *Local Adaptive Mesh Refinement for Shock Hydrodynamics P. COLELLA*. Tech. rep. 1989, p. 6484.
- [10] Marsha J Berger and Joseph Oliger. *Adaptive Mesh Refinement for Hyperbolic Partial Differential Equations**. Tech. rep. 1984, pp. 484–512.
- [11] Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. 1st ed. Springer, 2007. ISBN: 0387310738.
- [12] Steven L Brunton, Bernd R Noack, and Petros Koumoutsakos. "Machine Learning for Fluid Mechanics". In: *Annual Review of Fluid Mechanics Downloaded from www.annualreviews.org*. Guest 7 (2020), p. 29. DOI: [10.1146/annurev-fluid-010719](https://doi.org/10.1146/annurev-fluid-010719). URL: <https://doi.org/10.1146/annurev-fluid-010719>.
- [13] Andriy Burkov. *Machine Learning Engineering*. First edition. True Positive Inc., 2020. ISBN: 9781999579579. URL: <http://www.mlebook.com/>.
- [14] Jiayi Cai et al. "Revisiting Tensor Basis Neural Network for Reynolds stress modeling: Application to plane channel and square duct flows". In: *Computers and Fluids* 275 (May 2024). ISSN: 00457930. DOI: [10.1016/j.compfluid.2024.106246](https://doi.org/10.1016/j.compfluid.2024.106246).
- [15] *CFD Techniques and Thermo-Mechanics Applications*. Springer International Publishing, 2018. DOI: [10.1007/978-3-319-70945-1](https://doi.org/10.1007/978-3-319-70945-1).

- [16] Andrew Chapman, Yousef Saad, and Larry Wigton. "High-order ILU preconditioners for CFD problems". In: *International Journal for Numerical Methods in Fluids* 33 (2000), pp. 767–788.
- [17] John Y. Chen. "GPU technology trends and future requirements". In: *2009 IEEE International Electron Devices Meeting (IEDM)*. 2009, pp. 1–6. DOI: [10.1109/IEDM.2009.5424433](https://doi.org/10.1109/IEDM.2009.5424433).
- [18] Ricky T. Q. Chen et al. "Neural Ordinary Differential Equations". In: (June 2018). URL: <http://arxiv.org/abs/1806.07366>.
- [19] Ruilin Chen, Xiaowei Jin, and Hui Li. "A machine learning based solver for pressure Poisson equations". In: *Theoretical and Applied Mechanics Letters* 12 (5 Sept. 2022). ISSN: 20950349. DOI: [10.1016/j.taml.2022.100362](https://doi.org/10.1016/j.taml.2022.100362).
- [20] Clay Mathematics Institute. *Millennium Prize Problems: Navier-Stokes Equation*. Online; accessed 6 June 2025. <https://www.claymath.org/millennium/navier-stokes-equation>. 2000.
- [21] Gary Coleman and Richard Sandberg. "A Primer on Direct Numerical Simulation of Turbulence – Methods, Procedures and Guidelines". In: (Mar. 2010).
- [22] Thomas Daniel. "Machine learning for nonlinear model order reduction". 2021UPSLM039. PhD thesis. 2021. URL: <http://www.theses.fr/2021UPSLM039/document>.
- [23] James W. Deardorff. "A numerical study of three-dimensional turbulent channel flow at large Reynolds numbers". In: *Journal of Fluid Mechanics* 41.2 (1970), pp. 453–480. DOI: [10.1017/S0022112070000691](https://doi.org/10.1017/S0022112070000691).
- [24] James Donnelly, Alireza Daneshkhah, and Soroush Abolfathi. "Physics-informed neural networks as surrogate models of hydrodynamic simulators". In: *Science of the Total Environment* 912 (Feb. 2024). ISSN: 18791026. DOI: [10.1016/j.scitotenv.2023.168814](https://doi.org/10.1016/j.scitotenv.2023.168814).
- [25] Anshu Dubey et al. "A survey of high level frameworks in block-structured adaptive mesh refinement packages". In: *Journal of Parallel and Distributed Computing* 74 (12 Dec. 2014), pp. 3217–3227. ISSN: 0743-7315. DOI: [10.1016/J.JPDC.2014.07.001](https://doi.org/10.1016/J.JPDC.2014.07.001).
- [26] Hamidreza Eivazi et al. "Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations". In: *Physics of Fluids* 34.7 (July 2022), p. 075117. ISSN: 1070-6631. DOI: [10.1063/5.0095270](https://doi.org/10.1063/5.0095270). eprint: https://pubs.aip.org/aip/pof/article-pdf/doi/10.1063/5.0095270/19816179/075117_1_online.pdf. URL: <https://doi.org/10.1063/5.0095270>.
- [27] Hamidreza Eivazi et al. "Towards extraction of orthogonal and parsimonious non-linear modes from turbulent flows". In: *Expert Systems with Applications* 202 (Sept. 2022). ISSN: 09574174. DOI: [10.1016/j.eswa.2022.117038](https://doi.org/10.1016/j.eswa.2022.117038).
- [28] Olivier Esnault et al. "A method to accelerate LES explicit solvers using local time-stepping". In: *48th AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition*. American Institute of Aeronautics and Astronautics Inc., 2010. ISBN: 9781600867392. DOI: [10.2514/6.2010-123](https://doi.org/10.2514/6.2010-123).
- [29] Jenna Foale. "Simulating Extinction and Blow-off in Kerosene Swirl Spray Flames". PhD thesis. Oct. 2021. DOI: [10.17863/CAM.82714](https://doi.org/10.17863/CAM.82714).
- [30] Massimo Germano et al. "A Dynamic Subgrid-Scale Eddy Viscosity Model". In: *Physics of Fluids A* 3.7 (1991). Proposes the dynamic procedure for computing the Smagorinsky coefficient., pp. 1760–1765. DOI: [10.1063/1.857955](https://doi.org/10.1063/1.857955). URL: <https://doi.org/10.1063/1.857955>.
- [31] Kai Germaschewski et al. "Using Krylov-Schwarz methods in an adaptive mesh refinement environment". In: *Adaptive Mesh Refinement - Theory and Applications*. Ed. by Tomasz Plewa, Timur Linde, and V. Gregory Weirs. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 115–124. ISBN: 978-3-540-27039-3.
- [32] Paul Ghanem et al. *Learning Physics Informed Neural ODEs with Partial Measurements*. Tech. rep. 2025. URL: www.aaai.org.
- [33] M. Giovannini et al. "A Hybrid Parallelization Strategy of a CFD Code for Turbomachinery Applications". In: *Proceedings of the 11th European Conference on Turbomachinery Fluid Dynamics & Thermodynamics (ETC11)*. Open Access. Madrid, Spain: European Turbomachinery Society, Mar. 2015. URL: <https://www.euroturbo.eu/paper/ETC2015-118.pdf>.

- [34] Sharath S. Girimaji. *Turbulence closure modeling with machine learning: a foundational physics perspective*. July 2024. DOI: [10.1088/1367-2630/ad6689](https://doi.org/10.1088/1367-2630/ad6689).
- [35] Myeong Seok Go, Jae Hyuk Lim, and Seungchul Lee. "Physics-informed neural network-based surrogate model for a virtual thermal sensor with real-time simulation". In: *International Journal of Heat and Mass Transfer* 214 (Nov. 2023). ISSN: 00179310. DOI: [10.1016/j.ijheatmasstransfer.2023.124392](https://doi.org/10.1016/j.ijheatmasstransfer.2023.124392).
- [36] Volker Gravemeier et al. "An algebraic variational multiscale-multigrid method for large eddy simulation of turbulent flow". In: *Computer Methods in Applied Mechanics and Engineering* 199 (13-16 Feb. 2010), pp. 853–864. ISSN: 00457825. DOI: [10.1016/j.cma.2009.05.017](https://doi.org/10.1016/j.cma.2009.05.017).
- [37] Xiaoxiao Guo, Wei Li, and Francesco Iorio. "Convolutional Neural Networks for Steady Flow Approximation". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, California, USA: Association for Computing Machinery, 2016, pp. 481–490. ISBN: 9781450342322. DOI: [10.1145/2939672.2939738](https://doi.org/10.1145/2939672.2939738). URL: <https://doi.org/10.1145/2939672.2939738>.
- [38] Ioan Hadade and Luca di Mare. "Modern multicore and manycore architectures: Modelling, optimisation and benchmarking a multiblock CFD code". In: *Computer Physics Communications* 205 (Aug. 2016), pp. 32–47. ISSN: 00104655. DOI: [10.1016/j.cpc.2016.04.006](https://doi.org/10.1016/j.cpc.2016.04.006).
- [39] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: (Dec. 2015). URL: <http://arxiv.org/abs/1512.03385>.
- [40] Stefan Hickel et al. "A parametrized non-equilibrium wall-model for large-eddy simulations". In: (Nov. 2015). DOI: [10.13140/2.1.1281.6324](https://doi.org/10.13140/2.1.1281.6324). URL: <http://arxiv.org/abs/1511.09356> <http://dx.doi.org/10.13140/2.1.1281.6324>.
- [41] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. "Multilayer feedforward networks are universal approximators". In: *Neural Networks* 2.5 (1989), pp. 359–366. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8). URL: <https://www.sciencedirect.com/science/article/pii/0893608089900208>.
- [42] Md Lokman Hosain and Rebei Bel Fdhila. "Literature Review of Accelerated CFD Simulation Methods towards Online Application". In: *Energy Procedia*. Vol. 75. Elsevier Ltd, 2015, pp. 3307–3314. DOI: [10.1016/j.egypro.2015.07.714](https://doi.org/10.1016/j.egypro.2015.07.714).
- [43] Dana A. Jacobsen, Julien C. Thibault, and Inanc Senocak. "An MPI-CUDA implementation for massively parallel incompressible flow computations on multi-GPU clusters". In: *48th AIAA Aerospace Sciences Meeting Including the New Horizons Forum and Aerospace Exposition*. 2010. ISBN: 9781600867392. DOI: [10.2514/6.2010-522](https://doi.org/10.2514/6.2010-522).
- [44] Joongoo Jeon, Juhyeong Lee, and Sung Joong Kim. "Finite volume method network for the acceleration of unsteady computational fluid dynamics: Non-reacting and reacting flows". In: *International Journal of Energy Research* 46 (8 June 2022), pp. 10770–10795. ISSN: 1099114X. DOI: [10.1002/er.7879](https://doi.org/10.1002/er.7879).
- [45] Joongoo Jeon et al. "Residual-based physics-informed transfer learning: A hybrid method for accelerating long-term CFD simulations via deep learning". In: *International Journal of Heat and Mass Transfer* 220 (2024), p. 124900. ISSN: 0017-9310. DOI: <https://doi.org/10.1016/j.ijheatmasstransfer.2023.124900>. URL: <https://www.sciencedirect.com/science/article/pii/S0017931023010451>.
- [46] Chao Jiang et al. "An interpretable framework of data-driven turbulence modeling using deep neural networks". In: *Physics of Fluids* 33 (5 May 2021). ISSN: 10897666. DOI: [10.1063/5.0048909](https://doi.org/10.1063/5.0048909).
- [47] Haoqiang Jin et al. "High performance computing using MPI and OpenMP on multi-core parallel systems". In: *Parallel Computing* 37.9 (2011). Emerging Programming Paradigms for Large-Scale Scientific Computing, pp. 562–575. ISSN: 0167-8191. DOI: <https://doi.org/10.1016/j.parco.2011.02.002>. URL: <https://www.sciencedirect.com/science/article/pii/S0167819111000159>.

- [48] Nikhil Kalkote, Ashwani Assam, and Vinayak Eswaran. "Towards developing an adaptive time stepping for compressible unsteady flows". In: *International Journal of Numerical Methods for Heat and Fluid Flow* 29 (2 Feb. 2019), pp. 487–503. ISSN: 09615539. DOI: [10.1108/HFF-03-2018-0095](https://doi.org/10.1108/HFF-03-2018-0095).
- [49] A. Kolmogorov. "The Local Structure of Turbulence in Incompressible Viscous Fluid for Very Large Reynolds' Numbers". In: *Akademiia Nauk SSSR Doklady* 30 (Jan. 1941), pp. 301–305.
- [50] Marius Kurz, Philipp Offenhäuser, and Andrea Beck. "Deep reinforcement learning for turbulence modeling in large eddy simulations". In: *International Journal of Heat and Fluid Flow* 99 (Feb. 2023). ISSN: 0142727X. DOI: [10.1016/j.ijheatfluidflow.2022.109094](https://doi.org/10.1016/j.ijheatfluidflow.2022.109094).
- [51] Jianqi Lai et al. "Hybrid MPI and CUDA Parallelization for CFD Applications on Multi-GPU HPC Clusters". In: *Scientific Programming* 2020 (2020). ISSN: 10589244. DOI: [10.1155/2020/8862123](https://doi.org/10.1155/2020/8862123).
- [52] Johan Larsson et al. "Large eddy simulation with modeled wall-stress: recent progress and future directions". In: *Mechanical Engineering Reviews* 3 (1 2016), pp. 15-00418-15-00418. DOI: [10.1299/mer.15-00418](https://doi.org/10.1299/mer.15-00418).
- [53] B. E. Launder and D. B. Spalding. "The numerical computation of turbulent flows". In: *Computer Methods in Applied Mechanics and Engineering* 3 (2 Mar. 1974), pp. 269–289. ISSN: 0045-7825. DOI: [10.1016/0045-7825\(74\)90029-2](https://doi.org/10.1016/0045-7825(74)90029-2).
- [54] Brian E Launder, GJ Reece, and Wolfgang Rodi. "Progress in the development of a Reynolds-stress turbulence closure". In: *Journal of Fluid Mechanics* 68.3 (1975), pp. 537–566.
- [55] Sangseung Lee and Donghyun You. "Data-driven prediction of unsteady flow over a circular cylinder using deep learning". In: *Journal of Fluid Mechanics* 879 (2019), pp. 217–254. DOI: [10.1017/jfm.2019.700](https://doi.org/10.1017/jfm.2019.700).
- [56] J. Gordon Leishman. "Turbulent Flows". In: *Introduction to Aerospace Flight Vehicles*. Accessed: 2025-08-25. Eagle Pubs, Embry–Riddle Aeronautical University, 2023. URL: <https://eaglepubs.erau.edu/introductiontoaerospaceflightvehicles/chapter/turbulent-flows/>.
- [57] D. K. Lilly. "A Proposed Modification of the Germano Subgrid-Scale Closure Method". In: *Physics of Fluids A* 4.3 (1992). Refines Germano's dynamic model by introducing least-squares minimization to stabilize C_s , pp. 633–635. DOI: [10.1063/1.858280](https://doi.org/10.1063/1.858280). URL: <https://doi.org/10.1063/1.858280>.
- [58] Julia Ling, Andrew Kurzawski, and Jeremy Templeton. "Reynolds averaged turbulence modelling using deep neural networks with embedded invariance". In: *Journal of Fluid Mechanics* 807 (2016), pp. 155–166. DOI: [10.1017/jfm.2016.615](https://doi.org/10.1017/jfm.2016.615).
- [59] Rainald Löhner. "Improved error and work estimates for high-order elements". In: *International Journal for Numerical Methods in Fluids* 72 (11 Aug. 2013), pp. 1207–1218. ISSN: 02712091. DOI: [10.1002/flid.3783](https://doi.org/10.1002/flid.3783).
- [60] Lu Lu et al. "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators". In: *Nature Machine Intelligence* 3 (3 Mar. 2021), pp. 218–229. ISSN: 25225839. DOI: [10.1038/s42256-021-00302-5](https://doi.org/10.1038/s42256-021-00302-5).
- [61] M4 Engineering. *Simcenter STAR-CCM+ Adaptive Mesh Refinement*. Accessed: 2025-06-12. 2024. URL: <https://www.m4-engineering.com/simcenter-star-ccm-adaptive-mesh-refinement/>.
- [62] Peter MacNeice et al. "PARAMESH: A parallel adaptive mesh refinement community toolkit". In: *Computer Physics Communications* 126 (3 Apr. 2000), pp. 330–354. ISSN: 0010-4655. DOI: [10.1016/S0010-4655\(99\)00501-9](https://doi.org/10.1016/S0010-4655(99)00501-9).
- [63] Zhiping Mao, Ameya D. Jagtap, and George Em Karniadakis. "Physics-informed neural networks for high-speed flows". In: *Computer Methods in Applied Mechanics and Engineering* 360 (Mar. 2020). ISSN: 00457825. DOI: [10.1016/j.cma.2019.112789](https://doi.org/10.1016/j.cma.2019.112789).
- [64] Igor L. Markov. *Limits on fundamental limits to computation*. Aug. 2014. DOI: [10.1038/nature13570](https://doi.org/10.1038/nature13570).

- [65] J.A. Meijerink and H.A. van der Vorst. "An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix". In: *Mathematics of Computation* 31 (1977), pp. 148–162.
- [66] Davide Modesti and Stefan Hickel. *CFD for Aerospace Engineers: Practical Session 5 – Open Source Software*. Lecture slides, AE4202 CFD for Aerospace Engineers, TU Delft. Accessed: 2025-08-25. 2024.
- [67] Arvind Mohan et al. "Compressed Convolutional LSTM: An Efficient Deep Learning framework to Model High Fidelity 3D Turbulence". In: (Mar. 2019). URL: <http://arxiv.org/abs/1903.00033>.
- [68] Gordon E. Moore. "Cramming more components onto integrated circuits". In: *Electronics* 38.8 (Apr. 1965). Also known as "Moore's Law" paper, pp. 114–117.
- [69] F. Moukalled, L. Mangani, and M. Darwish. *The Finite Volume Method in Computational Fluid Dynamics : An Advanced Introduction with OpenFOAM® and Matlab*. Vol. 113. Fluid Mechanics and its Applications. Cham: Springer, 2016. ISBN: 978-3-319-16873-9. DOI: [10.1007/978-3-319-16874-6](https://doi.org/10.1007/978-3-319-16874-6).
- [70] NashTech Global. *Getting Familiar with Activation Function and Its Types*. <https://blog.nashtechglobal.com/getting-familiar-with-activation-function-and-its-types/>. Accessed: 2025-07-01. 2023.
- [71] F Nicoud and F Ducros. *Subgrid-Scale Stress Modelling Based on the Square of the Velocity Gradient Tensor*. Tech. rep. 1999, pp. 183–200.
- [72] FTM Nieuwstadt, Bendiks Jan Boersma, and Jerry Westerweel. *Turbulence: Introduction to Theory and Applications of Turbulent Flows*. English. Springer, 2016. ISBN: 978-3-319-31597-3. DOI: [10.1007/978-3-319-31599-7](https://doi.org/10.1007/978-3-319-31599-7).
- [73] M. R. Nived, Nikhil Narayan Kalkote, and Vinayak Eswaran. "Achieving rapid convergence in false-transient RANS simulations on unstructured meshes using an implicit adaptive time stepping method". In: *Computers and Fluids* 296 (June 2025). ISSN: 00457930. DOI: [10.1016/j.compfluid.2025.106644](https://doi.org/10.1016/j.compfluid.2025.106644).
- [74] Octavi Obiols-Sales et al. "CFDNet: A deep learning-based accelerator for fluid simulations". In: *Proceedings of the International Conference on Supercomputing*. Association for Computing Machinery, June 2020. ISBN: 9781450379830. DOI: [10.1145/3392717.3392772](https://doi.org/10.1145/3392717.3392772).
- [75] Pranshu Pant et al. "Deep learning for reduced order modelling and efficient temporal evolution of fluid simulations". In: *Physics of Fluids* 33 (10 Oct. 2021). ISSN: 10897666. DOI: [10.1063/5.0062546](https://doi.org/10.1063/5.0062546).
- [76] U. Piomelli. "Large eddy simulations in 2030 and beyond". In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 372 (2022 Aug. 2014). ISSN: 1364503X. DOI: [10.1098/rsta.2013.0320](https://doi.org/10.1098/rsta.2013.0320).
- [77] Ugo Piomelli. *Wall-layer models for large-eddy simulations*. Aug. 2008. DOI: [10.1016/j.paero sci.2008.06.001](https://doi.org/10.1016/j.paero sci.2008.06.001).
- [78] Tomasz Plewa, Timur Jaan Linde, Vincent Gregory Weirs, et al. "Adaptive mesh refinement, theory and applications". In: (2005).
- [79] Stephen B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.
- [80] P. Quéméré and P. Sagaut. "Zonal multi-domain RANS/LES simulations of turbulent flows". In: *International Journal for Numerical Methods in Fluids* 40 (7 Nov. 2002), pp. 903–925. ISSN: 02712091. DOI: [10.1002/flid.381](https://doi.org/10.1002/flid.381).
- [81] M. Raissi, P. Perdikaris, and G. E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378 (Feb. 2019), pp. 686–707. ISSN: 10902716. DOI: [10.1016/j.jcp.2018.10.045](https://doi.org/10.1016/j.jcp.2018.10.045).
- [82] Osborne Reynolds. "IV. On the dynamical theory of incompressible viscous fluids and the determination of the criterion". In: *Philosophical Transactions of the Royal Society of London. (A.)* 186 (1895), pp. 123–164. DOI: [10.1098/rsta.1895.0004](https://doi.org/10.1098/rsta.1895.0004). eprint: <https://royalsocietypublishing.org/doi/pdf/10.1098/rsta.1895.0004>. URL: <https://royalsocietypublishing.org/doi/abs/10.1098/rsta.1895.0004>.

- [83] Ferry Roelofs and A. Shams. “CFD—Introduction”. In: *Thermal Hydraulics Aspects of Liquid Metal Cooled Nuclear Reactors* (Jan. 2019), pp. 213–218. DOI: [10.1016/B978-0-08-101980-1.00006-5](https://doi.org/10.1016/B978-0-08-101980-1.00006-5).
- [84] Carlos J. G. Rojas, Andreas Dengel, and Mateus Dias Ribeiro. “Reduced-order Model for Fluid Flows via Neural Ordinary Differential Equations”. In: (Feb. 2021). URL: <http://arxiv.org/abs/2102.02248>.
- [85] Wybe Rozema et al. “Minimum-dissipation models for large-eddy simulation”. In: *Physics of Fluids* 27.8 (Aug. 2015), p. 085107. ISSN: 1070-6631. DOI: [10.1063/1.4928700](https://doi.org/10.1063/1.4928700). eprint: https://pubs.aip.org/aip/pof/article-pdf/doi/10.1063/1.4928700/15691134/085107_1_online.pdf. URL: <https://doi.org/10.1063/1.4928700>.
- [86] C L Rumsey and G N Coleman. *NASA Symposium on Turbulence Modeling: Roadblocks, and the Potential for Machine Learning*. Tech. rep. 2022. URL: <http://www.sti.nasa.gov>.
- [87] Benjamin Sanderse et al. *Scientific machine learning for closure models in multiscale problems: a review*. 2024. arXiv: [2403.02913](https://arxiv.org/abs/2403.02913) [math.NA]. URL: <https://arxiv.org/abs/2403.02913>.
- [88] Rafael Diez Sanhueza et al. “Machine learning for RANS turbulence modeling of variable property flows”. In: *Computers and Fluids* 255 (Apr. 2023). ISSN: 00457930. DOI: [10.1016/j.compfluid.2023.105835](https://doi.org/10.1016/j.compfluid.2023.105835).
- [89] U. Schumann. “Subgrid scale model for finite difference simulations of turbulent flows in plane channels and annuli”. In: *Journal of Computational Physics* 18 (4 Aug. 1975), pp. 376–404. ISSN: 0021-9991. DOI: [10.1016/0021-9991\(75\)90093-5](https://doi.org/10.1016/0021-9991(75)90093-5).
- [90] Martia Shahsavan, Mohammadrasool Morovatiyan, and John Mack. “A Computational Investigation of Non-Premixed Combustion of Natural Gas Injected Into Mixture of Argon and Oxygen”. In: Nov. 2018, V001T03A013. DOI: [10.1115/ICEF2018-9618](https://doi.org/10.1115/ICEF2018-9618).
- [91] John Shalf. *The future of computing beyond Moore’s Law*. Mar. 2020. DOI: [10.1098/rsta.2019.0061](https://doi.org/10.1098/rsta.2019.0061).
- [92] Varun Shankar et al. “Validation and parameterization of a novel physics-constrained neural dynamics model applied to turbulent fluid flow”. In: *Physics of Fluids* 34 (11 Nov. 2022). ISSN: 10897666. DOI: [10.1063/5.0122115](https://doi.org/10.1063/5.0122115).
- [93] Aleksei Sholokhov et al. “Physics-informed neural ODE (PINODE): embedding physics into models using collocation points”. In: *Scientific Reports* 13 (1 Dec. 2023). ISSN: 20452322. DOI: [10.1038/s41598-023-36799-6](https://doi.org/10.1038/s41598-023-36799-6).
- [94] Justin Sirignano and Jonathan F. MacArt. “Deep learning closure models for large-eddy simulation of flows around bluff bodies”. In: *Journal of Fluid Mechanics* 966 (July 2023). ISSN: 14697645. DOI: [10.1017/jfm.2023.446](https://doi.org/10.1017/jfm.2023.446).
- [95] J. Smagorinski. “GENERAL CIRCULATION EXPERIMENTS WITH THE PRIMITIVE EQUATIONS: I. THE BASIC EXPERIMENT”. In: *Monthly Weather Review* 91.3 (1963), pp. 99–164. DOI: [10.1175/1520-0493\(1963\)091<0099:GCEWTP>2.3.CO;2](https://doi.org/10.1175/1520-0493(1963)091<0099:GCEWTP>2.3.CO;2). URL: https://journals.ametsoc.org/view/journals/mwre/91/3/1520-0493_1963_091_0099_gcewtp_2_3_co_2.xml.
- [96] Paulo Sousa, Alexandre Afonso, and Carlos Veiga Rodrigues. “Application of machine learning to model the pressure poisson equation for fluid flow on generic geometries”. In: *Neural Computing and Applications* 36 (26 Sept. 2024), pp. 16581–16606. ISSN: 14333058. DOI: [10.1007/s00521-024-09935-0](https://doi.org/10.1007/s00521-024-09935-0).
- [97] P R Spalart. *Strategies for turbulence modelling and simulations*. URL: www.elsevier.com/locate/ijh.
- [98] P. R. Spalart and V. Venkatakrishnan. “On the role and challenges of CFD in the aerospace industry”. In: *Aeronautical Journal* 120 (1223 Jan. 2016). computational cost can be used in introduction of thesis, pp. 209–232. ISSN: 00019240. DOI: [10.1017/aer.2015.10](https://doi.org/10.1017/aer.2015.10).
- [99] Philippe Spalart. *AN OLD-FASHIONED FRAMEWORK FOR MACHINE LEARNING IN TURBULENCE MODELING*. Tech. rep.

- [100] Philippe R Spalart and Steven R Allmaras. “A one-equation turbulence model for aerodynamic flows”. In: *AIAA Paper* 92.439 (1992), pp. 1–22.
- [101] Charles G Speziale, Sutanu Sarkar, and Thomas B Gatski. “Modelling the pressure-strain correlation of turbulence: an invariant dynamical systems approach”. In: *Journal of Fluid Mechanics* 227 (1991), pp. 245–272.
- [102] N. A. Sreejith, Eleonore Riber, and Bénédicte Cuenot. “Analysis and design of a local time stepping scheme for LES acceleration in reactive and non-reactive flow simulations”. In: *Journal of Computational Physics* 470 (Dec. 2022). ISSN: 10902716. DOI: [10.1016/j.jcp.2022.111580](https://doi.org/10.1016/j.jcp.2022.111580).
- [103] Luning Sun et al. “Surrogate Modeling for Fluid Flows Based on Physics-Constrained Deep Learning Without Simulation Data”. In: (July 2019). DOI: [10.1016/j.cma.2019.112732](https://doi.org/10.1016/j.cma.2019.112732). URL: <http://arxiv.org/abs/1906.02382><http://dx.doi.org/10.1016/j.cma.2019.112732>.
- [104] TOP500 Project. *TOP500 Supercomputer Sites*. Accessed: 2025-06-09. 2025. URL: <https://www.top500.org>.
- [105] Eli Turkel. “Preconditioning Techniques in Computational Fluid Dynamics”. In: *Annual Review of Fluid Mechanics* 31 (1999), pp. 385–416.
- [106] Brian C Vermeire and Siva Nadarajah. “Adaptive IMEX time-stepping for the correction procedure via reconstruction scheme”. In: *21st AIAA Computational Fluid Dynamics Conference*. 2013, p. 2687.
- [107] Ricardo Vinuesa and Steven L. Brunton. “Enhancing computational fluid dynamics with machine learning”. In: *Nature Computational Science* 2 (6 June 2022), pp. 358–366. ISSN: 26628457. DOI: [10.1038/s43588-022-00264-7](https://doi.org/10.1038/s43588-022-00264-7).
- [108] A. W. Vreman. “The filtering analog of the variational multiscale method in large-eddy simulation”. In: *Physics of Fluids* 15 (8 2003). ISSN: 10706631. DOI: [10.1063/1.1595102](https://doi.org/10.1063/1.1595102).
- [109] Haixin Wang et al. “Recent Advances on Machine Learning for Computational Fluid Dynamics: A Survey”. In: (Aug. 2024). URL: <http://arxiv.org/abs/2408.12171>.
- [110] Jonathan M. Weiss and Wayne A. Smith. “Preconditioning applied to variable and constant density flows”. In: *AIAA Paper*. 1994-2209. 1994.
- [111] Gabriel D Weymouth and Bernat Font. *WaterLily.jl: A differentiable and backend-agnostic Julia solver for incompressible viscous flow around dynamic bodies*. Tech. rep. 2025. URL: <https://github.com/WaterLily-jl/WaterLily.jl>.
- [112] David C Wilcox. “Reassessment of the scale-determining equation for advanced turbulence models”. In: *AIAA Journal* 26.11 (1988), pp. 1299–1310.
- [113] Yue Xiang et al. “GPU Acceleration of CFD Algorithm: HSMAC and SIMPLE”. In: *Procedia Computer Science*. Vol. 108. Elsevier B.V., 2017, pp. 1982–1989. DOI: [10.1016/j.procs.2017.05.124](https://doi.org/10.1016/j.procs.2017.05.124).
- [114] Xiaoqiang Yue et al. “Parallelization of a DEM Code Based on CPU-GPU Heterogeneous Architecture”. In: *Parallel Computational Fluid Dynamics*. Ed. by Kenli Li et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2014, pp. 149–159. ISBN: 978-3-642-53962-6.
- [115] Yang Zhiyin. “Large-eddy simulation: Past, present and the future”. In: *Chinese Journal of Aeronautics* 28 (1 Feb. 2015), pp. 11–24. ISSN: 1000-9361. DOI: [10.1016/J.CJA.2014.12.007](https://doi.org/10.1016/J.CJA.2014.12.007).